APAPO

Universal Hypergrid Platform

Software Design Document

Attribute	Value
Document Type	Software Design Document (SDD)
System Name	Apapo — Universal Hypergrid Platform
Version	1.0
Date	March 2026
Status	Authoritative Reference — Active Design
Audience	Platform architects · System engineers · Product teams · Contributors
Domain Systems	Kogi (IW-OS) · Ume (B-OS) · Qala (SF-OS)
Substrate	Hypergrid N-Dimensional Distributed Spreadsheet System (NDSS)
Primary Languages	Rust (core substrate) · Go (services) · Scala 3 (engines) · TypeScript (SDK/UI) · SQL (PostgreSQL)
Module Codes	HG-CORE · HG-DIM · HG-CELL · HG-VIEW · HG-COMP · HG-CRDT · HG-GRAPH · HG-SPACE · HG-ID · HG-NS · HG-EXPORT · HG-AI · HG-QL · HG-PLUGIN · HG-OPS
Kogi Modules	KPMS · KPSS · KPRG · KPVW · KPCM · KPCL · KPEX · KLNK · KPID · KSPC · KWSP · KNSPC
Classification	Confidential — Internal Use Only

Table of Contents

Part I — Executive Overview

1. What is Apapo?

Apapo is the canonical name for the complete integrated platform ecosystem formed by the Hypergrid N-Dimensional Distributed Spreadsheet System (NDSS) substrate and the three domain operating systems that run on top of it: Kogi (Independent Worker OS), Ume (Business OS), and Qala (Solution Factory OS). Apapo is not a product name layered on top of these systems — it is the correct name for the convergent whole: the living, federated, AI-augmented platform that unifies how individuals, organizations, and solution factories manage their data, operations, intelligence, and economic relationships.

> *Core Insight: Apapo is built on a single radical insight — every structured domain entity, regardless of its domain, is a row in an N-dimensional distributed spreadsheet. A Kogi portfolio item, a Ume organizational module, a Qala solution — all are HyperRows in domain-specific Hypercubes sitting in the same Universal Cell Store, governed by the same CRDT logic, versioned by the same EventLog, connected by the same Hypergraph, and reasoned over by the same AI engine. Domain systems are lenses over this shared substrate.*

2. The Three Root Domains

Each of the three domain operating systems is organized around a single root domain concept — the entity that is the center of gravity for everything in that system:

System	Domain	Root Domain Concept	Root Hyperspreadsheet	Core Philosophical Claim
Kogi	Independent Worker OS	The Portfolio	Portfolio System — the master spreadsheet of the independent worker's entire operational world	Every entity in a worker's life — project, task, asset, gig, benefit, finance, relationship — is a row in one living portfolio spreadsheet.
Ume	Business OS	The Organization	Organization System — the master spreadsheet of the organization's entire operational world	Every entity in an organization's world — module, employee, legal entity, campaign, risk, OKR — is a row in one living organization spreadsheet.
Qala	Solution Factory OS	The Solution	Solution System — the master spreadsheet of the solution factory's entire production world	Every entity in a factory's world — SDE, artifact, CCR, release, toolchain, AI signal — is a row in one living solution spreadsheet. The factory itself is a Solution.

This root-domain architecture is not incidental — it is the primary design principle of the Apapo platform. Every sheet, every view, every AI signal, every governance workflow, every cross-system link in each domain system is organized in service of its root domain concept. The Portfolio is not a feature of Kogi — it is Kogi. The Organization is not a feature of Ume — it is Ume. The Solution is not a feature of Qala — it is Qala.

2.1 The Meta-Principle: Qala Governs Itself

The most philosophically powerful aspect of the Apapo architecture is Qala's self-referential root-domain principle: Qala (the Solution Factory OS) is itself a Solution. The Qala platform is a first-class HyperRow in its own qala.solutions Hypercube, governed by its own Root Factory, subject to its own CCR workflow, and tracked through its own lifecycle state. Furthermore, both Kogi and Ume are themselves Solutions that are governed within the Qala Solution ecosystem — they are produced by, versioned by, and released through Qala's governance framework. The platform governs itself through itself.

This self-describing architecture is not a toy property — it has practical consequences. The Qala operations team uses the full power of HyperQL and the Qala AI Agent to manage the Qala platform itself. Every Kogi feature release goes through a governed CCR. Every Ume module update is a Solution lifecycle event. The audit trail for the platform's own evolution is stored in the same Hypercubes that store customer solution audit trails.

3. Design Philosophy

Principle	Statement	Implementation
Universal Abstraction	Every platform entity — from a single task to a multi-national corporation — is representable as a HyperRow in a domain Hypercube	Single PortfolioComponent/OrgModule/Solution struct; typed discriminant; universal HyperCell schema superset
Root-Domain Sovereignty	Each domain system is organized entirely around its root concept. The Portfolio, the Organization, and the Solution are not features — they are the systems.	The root Hypercube in each system is the primary entity. Every other cube exists to enrich it.
Event-Sourced Truth	Every mutation is an immutable EventLog entry. Current state is always derivable from events. Nothing is ever deleted — only archived.	PostgreSQL append-only EventLog; CRDT replay on merge; snapshot + incremental replay for performance
Distributed-First	The system is designed for multi-node, multi-device, offline-capable operation from the ground up	VectorClock per component; LWW + OR-Set CRDT; federation via CrdtLog over Kafka; CrdtLog buffer in Redis

Principle	Statement	Implementation
Identity Sovereignty	A Sovereign Entity controls their data, their identities, their visibility, and their connections	KPID multi-identity; VisibilityMask; SplitPolicy; ProfilePartition; owner-level column encryption
Connected Economy	Every spreadsheet is a node in a global economic graph. Value and data flow through connections.	KLNK InterPortfolioLink; ShadowCell protocol; MirrorAttribute sync; LinkForest traversal
AI-Augmented Intelligence	The spreadsheet is not passive storage — it actively surfaces intelligence, flags risks, and suggests actions	kogi-engine WritebackService; AIColumn; Oba AI overlay; anomaly detection; pluggable AIEngineAdapter
Open Extensibility	The system is designed to be extended by users, organizations, and third-party developers at every layer	HypercubePlugin trait; FormulaColumn; custom column types; Domain Pack system; APPSTORE template marketplace
Self-Governing Platform	Qala governs the entire Apapo platform, including Kogi and Ume, as Solutions within its own root domain	Kogi and Ume are solutions in the Qala Root Factory. Platform releases go through CCR. Platform telemetry lives in Qala cubes.

Part II — Hypergrid: The N-Dimensional Distributed Spreadsheet Substrate

4. Overview and Formal Model

4.1 What is Hypergrid?

Hypergrid is a platform-agnostic, open-architecture, N-dimensional distributed spreadsheet system. It provides any application platform — productivity tools, project management systems, business intelligence platforms, domain operating systems, marketplaces, social networks — with a universal, infinitely extensible data substrate that models all entities, relationships, computations, and intelligence as a single, coherent, living grid.

A conventional spreadsheet has two dimensions: rows and columns. Hypergrid generalizes this to N dimensions: rows are Dimension 1, columns are Dimension 2, and any number of additional custom dimensions — time slices, geographic regions, organizational units, product versions, personas, risk categories, or any domain-specific axis — can be declared as Dimension 3 through N. Every cell in this N-dimensional grid is itself an N-attribute data point, carrying not just a single value but a structured map of typed attribute keys.

4.2 Formal Definition

```
Let a Hypercube H be defined by an ordered tuple of N dimension axes:

  H = (D₁, D₂, ..., D□)

  where:
    Dᵢ = DimensionAxis(id, name, type, key_set, ordering, cardinality)
    N  = the dimensionality of the Hypercube (N ≥ 2)

A HyperCell C is a data point addressed by a coordinate tuple:
  C = Cell(k₁ ∈ D₁.key_set, k₂ ∈ D₂.key_set, ..., k□ ∈ D□.key_set)

Each HyperCell carries an attribute map:
  C.attributes = { a₁: v₁, a₂: v₂, ..., a□: v□ }
  where aᵢ ∈ AttributeKeyRegistry and vᵢ is a TypedAttrValue

The Universal Cell Store (UCS) holds all HyperCells across all Hypercubes:
  UCS = { (grid_id, cube_id, dim_keys_tuple) → HyperCell }

A conventional 2D spreadsheet is the N=2 special case:
  D₁ = RowAxis    (entity dimension)
  D₂ = ColumnAxis (attribute/property dimension)
  C.attributes = { "value": V, "type": T, "format": F }

Domain entity storage convention (used by Kogi, Ume, Qala):
  D₁ = EntityAxis   (key_type: Uuid)          ← entity row (entity ID)
  D₂ = PropertyAxis (key_type: String)        ← property column (field name)
```

```
cell(entity_id, "field_name").attributes["value"] = <TypedAttrValue>
```

4.3 Conventional Spreadsheet vs Hypergrid

Concept	Traditional Spreadsheet	Hypergrid Generalization
Workbook	A single file containing multiple sheets	A Grid — the root container for all Hypercubes, tenants, namespaces, spaces, and graph structures
Sheet	A 2D grid of rows × columns	A Hypercube — an N-dimensional grid: $D_1 \times D_2 \times \ldots \times D_N$. Every Hypercube is a view/projection of the Universal Cell Store.
Row	A horizontal sequence of cells, first dimension	HyperRow — all cells sharing the same D_1 key. The canonical entity in a Hypercube.
Column	A vertical sequence of cells, second dimension	D_2 Axis — typed, indexed, governed. Each key is one property name. Per-key CRDT semantics and PermissionTier.
Extra dims	(none)	$D_3 \ldots D_N$ — custom, named, typed, indexed axes: Time | Geography | OrgUnit | Version | Tenant | Scenario | Hierarchy | Custom
Cell	A single value at the intersection of one row and one column	HyperCell — a data point at an N-dimensional coordinate, carrying an N-attribute map: value, type, formula, version, computed_by, visibility, tags, and any domain-defined key
Formula	A function over other cells in the same sheet	ComputedAttribute — Tier-1 (synchronous formula) or Tier-2 (asynchronous AI/ML signal) over any cell attribute across any dimension combination
Filter	A criterion applied to rows	DimSlice — a predicate applied to any combination of dimension axes, producing a sub-cube or flattened projection
Sheet tab	A named view over the same 2D grid	HypercubeView — a saved projection, filter, sort, group, and rendering configuration over any N-dimensional sub-space
Workbook link	An external reference to another workbook	CrossGridLink + ShadowCell — a typed, directed edge in the Hypergraph connecting entities across tenants, namespaces, or Grid deployments

5. Hypergrid Module Inventory

Module	Code	Language	Responsibility
Core Substrate	HG-CORE	Rust	Universal data model: Grid, Hypercube, DimensionAxis, HyperCell, HyperRow, EventLog, CrdtLog, VectorClock, PolicyEngine
Dimension System	HG-DIM	Rust	Declare, type, index, and govern custom dimensions. DimensionAxis registry. Sparse/dense/hybrid encoding. Cross-dim join planning.

Module	Code	Language	Responsibility
Cell Attribute Model	HG-CELL	Rust	N-attribute cell: attribute key registry, TypedAttrValue variants, attribute-level CRDT, computed attributes, inheritance, PermissionTier.
View Engine	HG-VIEW	Go	HypercubeView: N-dim filter (DimSlice), sort, group, pivot, projection, DimFold, DimExpand, board modes, rendering adapters.
Computation Engine	HG-COMP	Rust + Scala	ComputedAttribute evaluation: synchronous Tier-1 formula engine and asynchronous Tier-2 AI/ML engine. Rollup aggregation.
Distributed Consistency	HG-CRDT	Rust	N-dim CRDT: LWW per attribute, OR-Set for set-valued, MaxRegister, GrowOnlyCounter, Lattice for lifecycle. VectorClock per federation node.
Graph & Network	HG-GRAPH	Rust + Go	Hypergraph: typed directed edges between HyperCells, HyperRows, Cubes, Spaces, Grids. CrossGridLink, ShadowCell, LinkForest, BFS/DFS traversal.
Spaces & Workspaces	HG-SPACE	Go	Bounded operational contexts (Spaces) and active working sessions (Workspaces). GovernanceConfig. Treasury. SpaceLinkSubgraph.
Identity & Multi-Tenancy	HG-ID	Go	Multi-tenant, multi-identity, multi-profile model. TenantPartition, VisibilityMask, SplitPolicy, CrossTenantMerge, SovereignEntity.
Namespace System	HG-NS	Go	Hierarchical URI addressing for all entities. NamespacePath URI scheme. Federation-aware resolution protocol.
Export & Integration	HG-EXPORT	Go	CSV, XLSX, JSON, JSON-LD, Parquet, Apache Arrow, GraphQL, webhook, streaming export of any N-dim view.
AI Intelligence Layer	HG-AI	Rust + Python	Pluggable intelligence engines: anomaly detection, health scoring, recommendation, prediction, NL-to-HyperQL translation.
Query Language	HG-QL	Rust	HyperQL — N-dimensional query language: SELECT, DimSlice WHERE, FOLD, EXPAND, TRAVERSE GRAPH, AS_OF time-travel, SHADOW INCLUDE.
Plugin System	HG-PLUGIN	Rust	HypercubePlugin trait: custom dimension types, attribute types, AI engines, rendering adapters, data connectors, governance hooks.
Telemetry & Ops	HG-OPS	Go + Prometheus	Distributed tracing (OpenTelemetry), metrics, structured audit logs, health probes, federation sync health dashboards.

6. N-Dimensional Data Model

6.1 Dimension Axis Types

AxisType	Key Type	Ordering	Typical Use	Special Behaviors
EntityAxis	Uuid / String	Unordered / Custom	D_1 (rows) — primary entity dimension: users, projects, products, events, solutions	Full HyperRow model. GraphEdge::Hierarchy supported between keys. CRDT: LWW per attribute.
PropertyAxis	String	Lexicographic	D_2 (columns) — property/attribute dimension: field names, metric names, column labels	Column type registry. ComputedAttribute support. VisibilityMask per column key.
TimeAxis	Timestamp	Chronological	D_3 — temporal slicing: snapshots, time-series, event history, fiscal periods	AS_OF time-travel queries. Automatic snapshot generation. BRIN index. Rollup by hour/day/week/month/quarter/year.
GeoAxis	GeoHash / Coord	Spatial	D_4 — geographic slicing: region, country, city, custom polygon, lat/lng bucket	Spatial index (PostGIS). Drill-down: continent → country → region → city. Spatial join support.
CategoryAxis	String (enum)	Ordinal / Custom	$D_{3–N}$ — classification: product line, risk tier, department, scenario, metric category	Enum key validation. Hierarchy support. Cross-category pivot. Used in Qala solutions.metrics D_4.
HierarchyAxis	HierarchyPath	Tree (depth-first)	$D_{3–N}$ — org chart, product taxonomy, tag hierarchy, geographic admin levels	Parent-child key relationships. Rollup propagation. Drill-down/drill-up operations.
TenantAxis	TenantId	Unordered	D_3 — multi-tenancy: one Hypercube serving multiple tenants with per-key RLS	Row-level security: each key visible only to its tenant. Used for multi-tenant SaaS shared-cube deployments.
ScenarioAxis	String	Custom	$D_{3–N}$ — what-if and planning: base, optimistic, pessimistic, budget, forecast, actuals	Scenario comparison views. Variance computed across scenario keys. Plan-vs-actual analysis.
VersionAxis	SemVer / Integer	Numeric	D_3 — schema/data versioning across cube schema migrations and parallel versions	Fork-and-merge version operations. Diff between version keys. Schema migration tracking.
OrdinalAxis	Integer	Numeric	$D_{3–N}$ — ranked/ordered slicing: priority, ranking, version number, sequence	Dense numeric key space. Efficient range queries. Used for sprint sequences, ranking cubes.
GraphAxis	NodeId	Graph-order / Custom	$D_{3–N}$ — graph traversal dimension: network hops, dependency layers, influence tiers	KLNK/Hypergraph integration. Keys are graph node IDs. Enables adjacency matrix representations.

AxisType	Key Type	Ordering	Typical Use	Special Behaviors
PersonaAxis	String	Custom	D_3–N — audience/persona slicing: user segments, role types, maturity levels, view contexts	A/B testing support. Personalized ComputedAttributes. Feed personalization.
Custom	Plugin-defined	Plugin-defined	Any domain-specific axis not covered above: batch numbers, ingredient IDs, jurisdiction codes	HypercubePlugin provides key_type, validation, index, rendering, and computation handlers.

6.2 Dimensionality Constraints

Constraint	Value	Rationale
Minimum dimensions (N)	2	2D is the degenerate case: a conventional spreadsheet. All Hypergrid operations valid at N=2.
Maximum dimensions (N)	16 (hard limit)	Beyond 16 dims, query planning complexity exceeds practical utility. Recommended max: N=6 for most use cases.
Max keys per dense axis	10,000,000	Dense axes are fully materialized. Beyond 10M keys, use SparseAxis with on-demand materialization.
Max keys per sparse axis	Unlimited	Sparse axes store only cells with non-null attributes. Storage: O(non-null cells).
Max attributes per HyperCell	1,024	Attribute key registry limit per Grid. Practical range: 20–150 attributes per cell.
Max concurrent editors per cube	1,000	CRDT buffer capacity. Beyond 1K concurrent writers: federation node sharding recommended.
Max federation peers per Grid	255	VectorClock size. Each peer occupies one slot in the VectorClock HashMap<NodeId, u64>.

6.3 HyperCell Attribute Model

Attribute Key	TypedAttrValue	CRDT	PermissionTier	Description
value	Any TypedAttrValue	LWW	Member+	The primary cell value — the 'cell content' in spreadsheet terms
type	AttrType enum	LWW	Editor+	Declared type: Text | Number | Currency | Bool | DateTime | Enum | Json | Relation | Geo | Tag | User | Computed | Ai
formula	Option<FormulaExpr>	LWW	Editor+	If ComputedAttribute: the formula expression AST or AI engine reference

Attribute Key	TypedAttrValue	CRDT	PermissionTier	Description
version	u64	MaxRegister	System	Monotonic version counter incremented on every mutation
computed_by	Option<String>	LWW	System	If AI-computed: engine plugin ID and model version that produced this value
computed_at	Option<DateTime>	LWW	System	Timestamp of last AI computation
confidence	Option<f64>	LWW	System	AI confidence score [0.0–1.0] for AI-derived attributes
visibility	AttrVisibility	LWW	Owner	Public | Follower | Connection | Member | Owner — per-attribute visibility override
tags	Vec<String>	OR-Set	Member+	User-defined tags on this attribute value
crdt_semantics	CrdtSemantics	fixed at boot	System	LWW | OR-Set | GrowOnlyCounter | MaxRegister | Lattice — merge strategy for this attribute key
permission_tier	PermissionTier	LWW	Admin	Minimum permission to write: Public | Member | Editor | Manager | Admin | System
source_grid	Option<GridId>	LWW	System	If MirrorAttribute: the Grid that is the authoritative source. Read-only in host Grid.
last_actor	String	LWW	System	Identity string of the last actor to write this attribute (node_id:identity_tag)
last_mutated_at	DateTime<Utc>	LWW	System	Server-assigned timestamp of the last mutation
anomaly_flag	Option<AnomalyKind>	LWW (AI-write)	System	AI-detected anomaly: SuddenChange | PatternBreak | OutlierValue | StateChange

7. CRDT Distributed Consistency (HG-CRDT)

7.1 Per-Attribute CRDT Semantics

Hypergrid assigns CRDT semantics at the individual attribute key level — not at the record level or document level. This means different fields of the same entity can have different conflict resolution behavior, because different fields have semantically different properties. A 'name' field should resolve with last-write-wins (the most recent author wins). A 'tags' set should resolve with OR-Set semantics (both concurrent additions survive). A 'restart_count' should use a GrowOnlyCounter (it can only increase). A lifecycle status should follow a domain-specific lattice that enforces valid state transitions regardless of concurrent writes.

CRDT Type	Merge Rule	Suitable For	Apapo Examples
LastWriteWins (LWW)	max(timestamp_A, timestamp_B) → winning value	Any human-authored, single-valued field	name, description, title, config, JSON blobs, boolean flags, reference IDs
OR-Set	additions_A ∪ additions_B; removals apply only to causally-known entries	Multi-valued set fields	tags, owners, policy_ids, toolbox_ids, dependency_ids, member_lists, hashtags
GrowOnlyCounter	value = max(seen_max) or sum(distributed_increments)	Monotonically increasing integers	restart_count, view_count, like_count, contribution_count, link_count
MaxRegister	value = max(value_A, value_B)	Version numbers and monotonic maximums	version, sequence_number, sprint_number, semver components
Lattice (Lifecycle)	join(state_A, state_B) per partial order — no backward transitions	Governed lifecycle state fields	ComponentStatus, SolutionLifecycleState, ModuleLifecycleState, SpaceStatus

7.2 CRDT Selection Policy

Attribute Characteristic	CRDT Semantics	Rationale
Human-readable name, description, title	LastWriteWins	Text is authored by one person at a time; latest version wins semantically
Lifecycle status (Draft, Active, Archived)	LWW (v1); Lattice (v2)	LWW safe for v1; Lattice enforces governance invariants in v2
Multi-valued collections (tags, members, deps)	OR-Set	Concurrent additions must both survive; causally-safe removals
Monotonic counters (restart, view, engagement count)	GrowOnlyCounter	Counters only increase; max/sum merge always safe and semantically correct

Attribute Characteristic	CRDT Semantics	Rationale
Monotonic max values (version, sequence)	MaxRegister	Version numbers must never decrease; max merge is always correct
Binary flags (enabled, archived, locked)	LastWriteWins	Boolean flags have no concurrent conflict semantics beyond recency
JSON configuration blobs	LastWriteWins	Structured JSON treated as atomic; last writer owns the full config object
AI-computed signals (health, risk, match score)	LWW (system-write)	AI engines are sole writers; LWW on system-write is safe and auditable
Numeric accumulators (budget_spent, hours_logged)	GrowOnlyCounter	Summable values only increase via transactions; GrowOnly is semantically correct
Relationship references (parent_id, owner_id)	LastWriteWins	Reference fields have single-valued semantics; LWW is correct
Graph edge sets (children, links, dependents)	OR-Set	Edge sets are multi-valued; concurrent edge additions must both survive

7.3 VectorClock and Federation Protocol

Every Grid node maintains a VectorClock — a HashMap<NodeId, u64> that provides causal ordering of all mutations across federation peers. When two nodes merge their CrdtLogs, the VectorClock determines which operations are causally ordered (no conflict) vs. genuinely concurrent (require CRDT merge).

Federation sync operates over Apache Kafka via a delta-sync protocol. Each node publishes CrdtLog deltas to a per-Grid Kafka topic. Peer nodes subscribe to all connected Grid topics, apply incoming deltas through the HG-CRDT merge engine, and advance their VectorClocks. This protocol supports up to 255 federation peers per Grid (VectorClock slot limit). For larger deployments, hierarchical federation is used: Tier-1 Grids federate with Tier-2 Grids, which federate with their own peer sets.

Federation Tier	Protocol	Scope	Latency	Use Case
Within-Grid multi-node	CrdtLog delta over Kafka; VectorClock causal ordering	Per-attribute delta within one Grid's Hypercubes	< 100ms p99	Horizontal scaling within Kogi/Ume/Qala deployment
Cross-Grid CrossGridLink	MirrorAttribute delta sync over CrossGridLink channel; consent-gated	Per-approved-attribute delta between Grid deployments	< 1s near-real-time	Kogi ↔ Qala solution linking; Ume ↔ Kogi employer linking
Cross-Grid full federation	CrdtLog delta over Kafka Grid-to-Grid topic; full cube sync	Per-Hypercube delta between Grid deployments	< 1s near-real-time	Organizational Ume federation; multi-region Qala factory

Federation Tier	Protocol	Scope	Latency	Use Case
Bulk initial sync	Parquet snapshot export → replay on target	Full Hypercube snapshot for initial peer bootstrap	Minutes to hours (size-dependent)	New federation peer onboarding; disaster recovery restore

8. Hypergraph Layer (HG-GRAPH)

8.1 Overview

The Hypergraph is Hypergrid's property graph layer. It provides typed, directed, weighted edges between any two addressable entities in the system — HyperCells, HyperRows, Hypercubes, Spaces, or entire Grids. The Hypergraph is the connective tissue of Apapo: it models every relationship between entities across domain boundaries, tenant boundaries, and federation node boundaries. Every entity in every domain system is simultaneously a node in the shared Hypergraph.

8.2 Edge Type Taxonomy

EdgeType	Direction	Consent	Description	Apapo Uses
Hierarchy	Directed (parent → child)	No	Parent-child containment. Enables rollup and drill-down.	Program → Project → Task; Organization → Department → Team; Solution Factory → SDE → Artifact
Dependency	Directed (A depends on B)	No	A cannot complete before B. Cycle detection enforced at write.	SDE → Library; Project → Project; Module depends on Module
Collaborates	Undirected	Yes — mutual	Active collaboration. Requires consent flow from both parties.	Kogi worker ↔ Kogi worker; SDE team members; cooperative co-owners
InvestedIn	Directed (investor → asset)	Yes — from target	Economic investment or patronage.	Worker → Project; Org → Solution Factory; Angel → Cooperative
CrossGridLink	Directed (host → source)	Yes — from source	Inter-Grid reference. Creates ShadowCells in host Grid.	Kogi portfolio → Qala solution; Ume org → Kogi worker portfolio
Employs	Directed (org → worker)	Yes — from worker	Employment/contracting relationship.	Ume organization → Kogi worker; Factory → Developer
Produces	Directed (producer → output)	No	An entity or process produces a downstream artifact or entity.	SDE → Solution; Campaign → Lead; OKR → Initiative
Governs	Directed (policy → subject)	No	A governance rule or approval chain governs an entity.	Policy → Component; GovernanceProposal → Module; CCR → Solution
Association	Directed	No	Informational link without stronger semantic classification.	OKR cascades to Finance Module; Campaign uses Design Asset

EdgeType	Direction	Consent	Description	Apapo Uses
FederationPeer	Undirected	Grid-level trust	Two Grid deployments in a federation relationship.	Kogi Grid ↔ Ume Grid ↔ Qala Grid; Multi-org federation
Follows	Directed (A follows B)	No	A subscribes to B's public updates.	Worker follows portfolio; Organization follows solution releases
Endorses	Directed (A endorses B)	No	A vouches for a skill, work product, or entity.	Worker endorses another's skill; Organization endorses solution

8.3 ShadowCell and MirrorAttribute Protocol

When a CrossGridLink is created between Grid A (host) and Grid B (source), Hypergrid automatically creates ShadowCells in Grid A that reflect the linked entity from Grid B. ShadowCells are read-only in the host Grid and are updated by the MirrorAttribute sync protocol whenever the source entity in Grid B changes.

Protocol Step	Actor	Description
1. Link Request	Host Grid user	User in Grid A requests to link to an entity in Grid B. CrossGridLink record created with consent_status=Pending.
2. Consent Review	Source Grid user	User in Grid B reviews the request, specifies which attribute keys may be mirrored (mirrored_attrs), optionally sets write_back_attrs. Approves or rejects.
3. ShadowCell Creation	HG-GRAPH	On consent approval, Grid A creates a ShadowCell for the linked entity, populated with the approved MirrorAttribute values from Grid B.
4. Real-Time Delta Sync	HG-GRAPH (async)	When a mirrored attribute changes in Grid B, a delta is published to the CrossGridLink Kafka channel. Grid A's shadow sync consumer applies the delta to update the ShadowCell. Target latency: < 1s.
5. Optional Write-Back	HG-GRAPH (governed)	If write_back_attrs is configured, specific attributes from Grid A can be written back to Grid B's source entity, subject to Grid B's PermissionTier policy.
6. Link Revocation	Either party	Either party can revoke the link. ShadowCell transitions to shadow_status=Disconnected. Historical sync data preserved in EventLog.

8.4 LinkForest Architecture

The LinkForest is the complete set of all CrossGridLinks and Hypergraph edges rooted at a given Sovereign Entity — the full picture of their inter-entity network across all Grids. The LinkForest is organized as a collection of LinkTrees: each LinkTree is a rooted directed subgraph of the full Hypergraph from a single root entity to a configurable maximum depth

(default: depth 5). Link traversal APIs support BFS, DFS, shortest-path (Dijkstra with edge weights), community detection (Louvain), and centrality computation (betweenness, eigenvector). For graphs larger than 1M nodes, sampled random-walk approximation is used for centrality.

9. Spaces, Workspaces, and Namespaces (HG-SPACE · HG-NS)

9.1 The Space System

A Space is a named, governed, bounded operational context within a Grid. It is itself a HyperRow — a first-class entity in a Space registry Hypercube — that groups a collection of Hypercubes, a member roster with tiered roles, governance configuration, and optionally a treasury. Spaces are the organizational layer that structures how portfolios, identities, organizations, and factories are scoped, governed, and discovered.

SpaceType	Description	Default Governance	Portfolio/Org Type	Namespace Prefix
Personal	Single user's private environment. Created automatically on registration.	Owner only	Personal Portfolio	@{handle}/
Team	Small collaborative group. Members share a project portfolio.	Team lead + member consensus	Shared Project Portfolio	team/{name}/
Organization	Formal organization (LLC, Corp, Trust, Coop, DAO).	Org governance model (varies)	Organization Portfolio	org/{slug}/
Cooperative	Member-owned with democratic governance and shared treasury.	1-member-1-vote	Cooperative Portfolio	coop/{slug}/
Collective	Open-contribution with light moderation.	Consensus / steward review	Collective Portfolio	collective/{slug}/
Community	Public or semi-public topical community / professional association.	Admin + community vote	Community Portfolio	community/{slug}/
Federation	Meta-Space linking orgs, coops, and collectives for joint governance.	Federal / representative	Federation Portfolio	fed/{slug}/
Project	Time-bounded Space tied to one project lifecycle.	Project owner	Project Portfolio	project/{id}/
Research	Research collaborations: academic, market, R&D.	PI / research lead	Research Portfolio	research/{slug}/
Event	Time-bounded Space for a specific event. Auto-archives after event.	Event host	Event Portfolio	event/{id}/
Public Studio	Creator's public-facing Space for showcasing work and monetizing content.	Creator / owner	Creator Portfolio	studio/{handle}/

SpaceType	Description	Default Governance	Portfolio/Org Type	Namespace Prefix
Factory	Qala Solution Factory operational context.	Factory governance pack	Solution System	factory/{slug}/

9.2 Space Member Roles

Role	PermissionTier	Capabilities
Owner	Owner (6)	Full control: configure Space, manage all members, manage treasury, dissolve Space, manage namespaces, all content and governance
Admin	Admin (7)	Platform-level super-admin override. Reserved for platform administrators.
Governor	Manager (5)	Submit and vote on governance proposals, manage treasury within approved limits, configure Space policy
Steward	Manager (5)	Review and approve contributions, moderate content, manage member roles below Steward level
Treasurer	Manager (5)	Manage treasury operations: initiate distributions, approve expense claims, manage linked bank account
Editor	Editor (4)	Create and edit Space portfolio components, create Workspaces, manage Space views and templates
Contributor	Contributor (3)	Submit contributions to shared portfolio, comment on proposals, participate in polls
Member	Member (2)	Standard Space membership: access shared Workspaces, view shared sheets, participate in Rooms
Viewer	Viewer (1)	Read-only access to Space public/internal content. Cannot contribute or join Rooms.
Guest	Viewer (1)	Temporary read-only access. No contribution rights. Access expires automatically.
Bot	System	Automated agent (Oba, integration bot). Configurable capability set defined per bot registration.

9.3 Namespace System

The Namespace System (HG-NS) provides hierarchical, globally unique URI addressing for every entity in the Apapo platform. Every entity — Grid, Space, Hypercube, HyperRow, HyperCell — has a NamespacePath URI resolvable across all federated Grid deployments.

Entity Type	NamespacePath Pattern	Example
Grid	hypergrid://{grid_slug}/	hypergrid://kogi-platform/
Space	hypergrid://{grid}/{space_type}/{slug}/	hypergrid://kogi-platform/personal/alice/

Entity Type	NamespacePath Pattern	Example
Hypercube	hypergrid://{grid}/{space}/{cube_name}/	hypergrid://kogi-platform/personal/alice/components/
HyperRow (Entity)	hypergrid://{grid}/{space}/{cube}/{entity_id}/	hypergrid://kogi-platform/personal/alice/components/proj-uuid/
HyperCell	hypergrid://{grid}/{space}/{cube}/{entity_id}/{field}	hypergrid://kogi-platform/personal/alice/components/proj-uuid/health_score
Kogi worker	kogi://{user_handle}/	kogi://@alice_creator/
Kogi portfolio	kogi://portfolio/{portfolio_slug}/	kogi://portfolio/alice-design-studio/
Kogi cooperative	kogi://coop/{coop_slug}/	kogi://coop/pamoja-tech-coop/
Ume organization	ume://{org_slug}/	ume://acme-corp/
Ume module	ume://{org_slug}/modules/{module_id}/	ume://acme-corp/modules/finance-accounting/
Qala factory	qala://factory/{factory_slug}/	qala://factory/pharma-factory/
Qala SDE	qala://factory/{factory_slug}/sde/{sde_id}/	qala://factory/pharma-factory/sde/drug-app-v1/
Qala solution	qala://factory/{factory_slug}/solutions/{sol_id}/	qala://factory/pharma-factory/solutions/tablet-formula-v3/

10. HyperQL — N-Dimensional Query Language (HG-QL)

10.1 Language Overview

HyperQL is Hypergrid's native query language for N-dimensional data. It extends SQL-like syntax with operations specific to N-dimensional grids: DimSlice (multi-axis filtering), DimFold (axis aggregation), DimExpand (axis pivot), TRAVERSE GRAPH (Hypergraph traversal), and AS_OF (time-travel). HyperQL is the primary query interface for all domain systems, analytical models, and AI engines.

10.2 Core Operations Reference

Operation	Syntax	Description	Generalization of
SELECT	SELECT cell[D_1, D_2].value AS field FROM {cube}	Select attribute values from cells at specified dimension coordinates	SQL SELECT
WHERE (DimSlice)	WHERE D_1.id = {val} AND D_3.period = '2026-Q2'	Filter cells by predicates on any combination of dimension axes	SQL WHERE / OLAP DimSlice
FOLD D_n	FOLD D_3 WITH SUM(value)	Collapse a dimension axis by aggregating all its key values into a summary	SQL GROUP BY / OLAP Roll-up
EXPAND D_n	EXPAND D_4 AS COLUMNS(AVG(value))	Expand a dimension's key set as separate result columns	SQL PIVOT
TRAVERSE GRAPH	TRAVERSE GRAPH FROM {id} EDGE_TYPE=Hierarchy MAX_DEPTH=5	Traverse the Hypergraph from a starting node, returning matching entities	Graph BFS/DFS query
AS_OF	SELECT * FROM {cube} AS_OF '2026-03-15T14:22:00Z'	Return cube state at the specified historical timestamp via EventLog replay	Event sourcing point-in-time query
SHADOW INCLUDE	INCLUDE SHADOW CELLS FROM hypergrid://partner/	Include ShadowCells from a linked CrossGridLink Grid in results	Federated query / external table
JOIN (GRAPH)	JOIN qala.solutions ON GRAPH_EDGE(a.id, b.id, 'CrossGridLink')	Join two cubes via a Hypergraph edge predicate	SQL JOIN with graph semantics
ROLLUP	ROLLUP(D_5, attr='budget_allocated')	Bottom-up hierarchical aggregation: leaves sum to parent nodes	SQL ROLLUP / OLAP drill-up

10.3 HyperQL Examples

```
-- 1. Basic entity query (N=2)
SELECT cell[D₁.id, "name"].value, cell[D₁.id, "health_score"].value
FROM kogi.portfolio.components
WHERE cell[D₁.id, "status"].value = '"Active"'
ORDER BY cell[D₁.id, "health_score"].value DESC
LIMIT 20;

-- 2. Time-series slice (N=3 with TimeAxis)
SELECT D₁.entity_id, D₃.period, cell[D₁, D₂, D₃, "value"].value AS metric_value
FROM kogi.portfolio.kpis
WHERE D₂.metric_name = 'health_score'
  AND D₃.period BETWEEN '2025-Q1' AND '2026-Q4'
ORDER BY D₁.entity_id, D₃.period;

-- 3. DimFold: collapse time axis by averaging
SELECT D₁.entity_id, FOLD D₃ WITH AVG(cell.value) AS avg_score
FROM kogi.portfolio.kpis
WHERE D₂.metric_name = 'health_score'
  AND D₃.period >= '2025-Q1'
GROUP BY D₁.entity_id;

-- 4. Graph traversal: drill down hierarchy from a program
TRAVERSE GRAPH
  FROM {program_id}
  EDGE_TYPE = Hierarchy
  DIRECTION = Outbound
  MAX_DEPTH = 3
SELECT node.id, node.name, node.status, node.health_score;

-- 5. AS_OF time-travel
SELECT cell[D₁, "name"].value, cell[D₁, "lifecycle_state"].value
FROM qala.solutions
AS_OF '2026-01-01T00:00:00Z'
WHERE cell[D₁, "factory_id"].value = '"root-factory-uuid"';

-- 6. Cross-cube join via Hypergraph (Kogi ↔ Qala)
SELECT a.name AS portfolio_item, b.name AS linked_solution, b.lifecycle_state
FROM kogi.portfolio.components AS a
JOIN qala.solutions AS b ON GRAPH_EDGE(a.id, b.id, 'CrossGridLink')
WHERE a.status = '"Active"' AND b.lifecycle_state IN ('"Active"', '"Released"');

-- 7. 4-dim quality pivot: Qala solution metrics by category over time
SELECT D₁.solution_id,
       EXPAND D₄ AS COLUMNS(AVG(cell.value))
FROM qala.solutions.metrics
WHERE D₂.metric_name IN ('quality_score', 'defect_density', 'security_score')
  AND D₃.period >= '2026-Q1'
GROUP BY D₁.solution_id;
-- Result: solution_id | security_avg | quality_avg | compliance_avg

-- 8. Cross-system query: workers in cooperative contributing to Qala solutions
SELECT D₁.worker_id
FROM kogi.portfolio.components
WHERE D₂."tags" CONTAINS 'cooperative:pamoja-tech'
  AND D₂."status".value = '"Active"';
TRAVERSE GRAPH FROM (above)
  EDGE_TYPE = CrossGridLink  DIRECTION = Outbound  TARGET_GRID = qala://
SELECT target.solution_id, target.name, target.lifecycle_state;
```

11. AI Intelligence Layer (HG-AI)

11.1 Two-Tier Computation Model

Tier	Name	Execution Model	Latency	Examples
Tier 1	Synchronous Formula	Evaluated inline on every read or write, in the HG-COMP formula engine. Deterministic.	< 1ms	budget_remaining = allocated - spent; health_score = weighted_avg(kpi_scores); progress_pct = completed_tasks / total_tasks
Tier 2	Asynchronous AI Signal	Triggered by EventLog mutations. Computed asynchronously by registered AIEngineAdapter plugins. Written back via WritebackService.	100ms – 30s	PortfolioHealthScore, RiskScore, AnomalyFlag, MatchScore, DriftStatus, RegulatoryRiskScore, NarrativeSummary

11.2 AIEngineAdapter Plugin Interface

```
pub trait AIEngineAdapter: Send + Sync {
    fn engine_id(&self) -> &str;   // e.g. "kogi.health_score_engine.v2"
    fn name(&self) -> &str;
    fn version(&self) -> &str;

    // Which attribute keys does this engine produce?
    fn output_keys(&self) -> Vec<AttributeKey>;

    // Which attribute keys does this engine consume as inputs?
    fn input_keys(&self) -> Vec<AttributeKey>;

    // Compute AI-derived attributes for a given HyperRow context.
    // Context provides: current_attrs, historical_events, graph_neighbors,
cross_cube_refs
    async fn compute(
        &self,
        ctx: &AiComputeContext<'_>,
    ) -> HypergridResult<Vec<(AttributeKey, TypedAttrValue)>>;

    // Called when an input attribute changes — should we trigger recomputation?
    fn should_recompute(
        &self,
        changed_key: &AttributeKey,
        old_value: &TypedAttrValue,
        new_value: &TypedAttrValue,
    ) -> bool;

    // Decay policy: how long before an AI-computed value is considered stale?
    fn staleness_ttl(&self) -> Duration;
}
```

11.3 AI Writeback Protocol

When an AI engine completes computation, it submits results to the WritebackService — a gRPC service that accepts engine-computed values, validates permissions, enforces audit trails, and writes TypedAttrValues into HyperCells. The WritebackService enforces:

- Only PermissionTier::System actors can write to AI-computed attribute keys
- Every AI write is recorded in the EventLog with actor='ai::{engine_id}:{model_version}'
- computed_by, computed_at, and confidence attributes are always co-written alongside value
- Stale AI signals are invalidated after the configured staleness_ttl if no recomputation is triggered

11.4 Natural Language to HyperQL Bridge

HG-AI includes a NLToHyperQLEngine plugin that translates natural language queries into valid HyperQL statements. The NL engine is initialized with each domain system's Hypercube registry, making it domain-vocabulary-aware: it understands 'project' as kogi.portfolio.components, 'module' as ume.kernel.modules, and 'SDE' as qala.sdes, translating accordingly.

System	Natural Language Input	Generated HyperQL (abbreviated)
Kogi	Show me everything at risk this week	SELECT * FROM kogi.portfolio.components WHERE risk_score > 70 OR (due_date < TODAY+7 AND status='Active')
Kogi	Which of my projects are over budget this quarter?	SELECT name, budget_utilization_pct FROM components WHERE type='Project' AND D_3.period=current_quarter AND budget_utilization_pct > 100
Ume	Which departments have the highest attrition risk?	SELECT dept, AVG(attrition_risk_score) FROM ume.hr.employees JOIN ume.analytics.kpis ON dept GROUP BY dept ORDER BY avg DESC
Qala	Which SDEs have had the most drift events in 90 days?	SELECT sde_id, COUNT(*) AS drift_count FROM qala.sdes WHERE drift_status != 'Clean' AND updated_at > NOW()-90d GROUP BY sde_id ORDER BY drift_count DESC
Qala	List solutions ready for release in the pharma factory	SELECT name, quality_score, compliance_score FROM qala.solutions WHERE lifecycle_state='Approved' AND factory_id='pharma-factory' AND release_readiness > 0.9

12. Security, Governance, and Compliance

12.1 Permission Tier Model

Permission Tier	Int	Kogi	Ume	Qala	Can Write
Public	0	Anonymous read	Anonymous read	Public solution browsing	No — read-only for Public-visibility attributes only
Viewer	1	Follow/watch portfolio	Employee read-only	SDE observer	No write access
Subscriber	1	Subscribe to updates	Subscribe to module alerts	Receive release notifications	No write access
Member	2	Portfolio collaborator	Org member	SDE team member	Standard user fields: description, tags, status on own rows
Contributor	3	Portfolio contributor	Record submitter	Junior developer	Content fields on assigned rows; comment and annotation fields
Editor	4	Portfolio editor	Domain record editor	Developer with SDE access	All Member fields + configuration, schema, timeline, budget fields
Manager	5	Portfolio manager	Department head	Tech Lead / Release Manager	All Editor fields + governance, member roster, Space settings
Admin	6	Platform Admin	Ume Kernel Admin	Factory Admin	All fields including governance records and audit configuration
System	7	AI engine / sync agent	Kernel services	AI Agent / CI pipeline	AI-computed attributes, federation sync fields, system metadata

12.2 EventLog as Immutable Audit Trail

Every mutation to any HyperCell attribute in the Universal Cell Store produces an EventLog entry. The EventLog is append-only and cryptographically hash-chained (optional) — entries can never be modified or deleted. Every entry records: actor identity, timestamp, before-value, after-value, CRDT operation applied, and VectorClock state.

For Qala's compliance requirement (generating compliance reports in under 4 hours, vs. 4–12 weeks industry average), this means: every CCR approval, every solution release, every SDE mutation is an EventLog entry. Compliance reports are generated by HyperQL AS_OF queries against the EventLog filtered by time range and event kind. For regulated industries, the EventLog can be exported with hash-chain verification to a separate compliance vault.

12.3 GDPR/CCPA Automation

Compliance Requirement	Hypergrid Mechanism
Right to Erasure (GDPR Art. 17)	SovereignEntity-level archive + personal data attribute tombstoning. EventLog retains the structural record; attribute values replaced with {GDPR_ERASED} markers.
Data Portability (GDPR Art. 20)	HG-EXPORT Parquet/JSON-LD export of all Hypercubes owned by an identity, including full EventLog and complete attribute history. Machine-readable and human-readable formats.
Consent Tracking	CrossGridLink consent records stored as immutable HyperRows: consent timestamp, consented attribute list, consent withdrawal timestamp, and withdrawing actor.
Data Residency	GeoAxis on TenantAxis enforces that certain Hypercubes are only stored on federation nodes in specified geographic regions (GDPR adequacy regions, sovereignty zones).
Audit Trail Tamper-Evidence	Optional cryptographic hash chaining on EventLog. Each entry's hash includes the previous entry's hash. Hash chain provides tamper-evidence for regulatory submissions.
Purpose Limitation (GDPR Art. 5c)	AttrVisibility masks enforce that each attribute key is accessible only to its declared purpose (owner read, AI computation, federation sync, public read).

Part III — Kogi: Independent Worker Operating System

13. Kogi Platform Overview

13.1 The Portfolio as Root Domain

Kogi's root domain concept is the Portfolio. The Portfolio is not a feature, a module, or a view — it is the central organizing principle of the entire platform. Every entity in the Kogi ecosystem — program, project, task, artifact, resource, gig, benefit, financial record, governance action, analytics event, contact, marketplace listing — is a row in the Portfolio. The Portfolio System (KPMS) is the master spreadsheet of the independent worker's entire operational and creative world.

> *Design Principle: Every entity in the Kogi ecosystem is a row. Every property is a column. Every perspective is a sheet. Every connection is an edge. Every boundary is a Space. The KIMDSS is not a feature — it is the operating system layer on which all Kogi applications are built.*

13.2 Kogi Module Architecture

Module Code	Module Name	Language	Responsibility
KPMS	Portfolio Management System	Go + Rust	Master orchestration layer: spreadsheet substrate, row/column model, sheet registry, view engine
KPSS	Portfolio Spreadsheet Substrate	Rust	Low-level data engine: PortfolioRow, ColumnSchema, SheetDefinition, indexed CellStore, formula engine
KPRG	Portfolio Component Registry	Go + PostgreSQL	Canonical registry of all PortfolioComponents system-wide; the master index across all sheets
KPVW	View Engine	Go	Filter · Sort · Group · Pivot · Slice across any column dimension; 30+ board modes; ChartView; PivotView
KPCM	Computation Engine	Rust + Scala	20+ analytical models; derived column computation; rollup and aggregation; kogi-engine integration
KPCL	Collaborative Editing	Rust	CRDT-backed concurrent multi-user spreadsheet editing; contribution attribution; conflict resolution UI
KPEX	Export & Integration	Go	CSV · XLSX · JSON-LD · Parquet · API · Embed · Webhook export of any sheet or view

Module Code	Module Name	Language	Responsibility
KLNK	Link Network	Go + Rust	Inter-portfolio graph: forests, trees, ShadowRows, MirrorColumns, cross-user connections; consent flows
KPID	Portfolio Identity System	Go	Multi-account, multi-identity, multi-profile management; VisibilityMask; SplitPolicy; ProfilePartition
KSPC	Space Module	Go	Named, governed, bounded digital environments (Personal, Team, Org, Coop, Community, Federation, Event, Studio)
KWSP	Workspace Module	Go	Active working context within a Space: open sheets, session state, tool layout, presence, session history
KNSPC	Namespace Module	Go	Hierarchical URI addressing for all platform entities via NamespacePath; platform DNS equivalent

13.3 Five-Layer + Extended Architecture

Layer	Name	Components & Responsibility
L0 — Substrate	Raw Component Store	PortfolioComponent (Rust) · ComponentMetadata · ComponentData · EventLog · CrdtLog · VectorClock · GraphEdge · PolicyEngine · ToolBoxId refs
L1 — Registry	Component Index	KPRG Master Registry · ComponentIndex · OwnerIndex · TagIndex · TypeIndex · StatusIndex · FullTextIndex · LinkNetworkIndex
L2 — Spreadsheet	KPSS Row/Column/Cell	PortfolioRow · ColumnSchema · SheetDefinition · CellStore · ColumnComputer · AggregationEngine · ViewFilter/Sort/Group
L3 — View Engine	KPVW View Layer	ViewFilter · ViewSort · ViewGroup · PivotView · ChartView · RowHighlightRule · PinnedColumns · BoardMode · SavedViews · ViewTemplate
L4 — Intelligence	kogi-engine Signals	ComputedColumns · HealthScoreColumn · RiskScoreColumn · MatchScoreColumn · RecommendationColumn · AnomalyFlagColumn · Oba AI overlay
L5 — Applications	User-facing Apps	kogi-portfolio UI · kogi-office Boards · kogi-bank Finance Views · kogi-marketplace Asset Views · kogi-community Feed Integration · KPEX
L6 — Link Network	KLNK Graph	InterPortfolioLink · LinkForest · LinkTree · VisibilityProtocol · SubscriptionEdge · FederationEdge · ShadowRow · MirrorColumn
L7 — Identity	KPID Multi-Identity	RootSpreadsheet · AccountMapping · ProfilePartition · IdentityAnchor · CrossIdentityMerge · VisibilityMask · SplitPolicy

14. Portfolio Core Data Model

14.1 PortfolioComponent — The Universal Node

The PortfolioComponent is the universal node of the entire KIMDSS. Every row in every sheet, every Space, every Workspace, every sheet definition, every link node — is a PortfolioComponent. It is either an Item (a leaf work entity: Task, Gig, Artifact, Metric) or a Container (an organizing structure: Program, Project, Portfolio, Space, Workspace). The Rust struct definition is the authoritative schema:

```
// kogi/portfolio_system.rs — Core Component struct
pub struct Component {
    pub metadata:  ComponentMetadata,
    pub data:      ComponentData,
    pub item_book: Option<ItemBookData>,  // typed payload for each ItemType
}

pub struct ComponentMetadata {
    pub id:            ComponentId,      // UUID — globally unique, immutable
    pub owners:        Vec<UserId>,
    pub tags:          HashSet<String>,
    pub policy_ids:    Vec<PolicyId>,
    pub created_at:    DateTime<Utc>,
    pub updated_at:    DateTime<Utc>,
    pub last_actor:    String,           // node_id:identity_tag for CRDT tiebreak
    pub vector_clock:  VectorClock,
    pub properties:    HashMap<String, serde_json::Value>,
    pub version:       VersionString,    // semver
    pub budget:        f64,
    pub budget_spent:  f64,
    pub resource_units:f64,
    pub namespace_path:Option<NamespacePath>,  // KNSPC address
    pub space_id:      Option<ComponentId>,    // parent Space (KSPC)
    pub workspace_ids: Vec<ComponentId>,       // open Workspaces (KWSP)
    pub identity_tags: Vec<IdentityTag>,       // KPID partition tags
}

pub struct ComponentData {
    pub category:       ComponentCategory,  // Item | Container
    pub name:           String,
    pub description:    String,
    pub status:         ComponentStatus,
    pub state:          ComponentState,
    pub visibility:     Visibility,
    pub children:       Vec<ComponentId>,
    pub parents:        Vec<ComponentId>,
    pub links:          Vec<ComponentId>,
    pub dependents:     Vec<ComponentId>,
    pub dependencies:   Vec<ComponentId>,
    pub users:          ComponentUsers,
    pub analytics:      ComponentAnalytics,
    pub risks:          Vec<Risk>,
    pub hashtags:       HashSet<String>,
    pub topics:         HashSet<String>,
    pub toolbox_ids:    Vec<ToolBoxId>,
    pub plugin_configs: HashMap<String, serde_json::Value>,
    pub space_context:  Option<SpaceContext>,  // KSPC integration
    pub link_metadata:  Option<LinkMetadata>,  // KLNK integration
}
```

14.2 Component Type Taxonomy

ItemType	Description	Typical Parent	Key Fields
Portfolio	The root container — the worker's master portfolio	Personal Space	name, description, owner, visibility, portfolio_type
Program	Strategic initiative grouping multiple projects	Portfolio	mission, objectives, budget_total, kpi_targets, status
Project	Bounded effort with deliverables and timeline	Program	methodology, sprints, releases, budget, due_date, progress_pct
Task / Story / Epic	Atomic or composite unit of work	Project or Sprint	priority, story_points, assignee, sprint_id, acceptance_criteria
Artifact	Versioned output: document, design, codebase, report	Project or Resource	artifact_type, version, format, signed_by, hash, file_url
Resource	Reusable asset: person, tool, license, service	Program or Portfolio	resource_type, capacity, cost_rate, availability, skills
Asset	Capital asset: IP, financial asset, digital asset	Portfolio	asset_type, valuation, acquisition_cost, depreciation_schedule
Gig / Contract	Contracted income-generating engagement	Program or Resource	platform, rate, currency, deliverables, client_id, payment_status
Benefit Account	Coverage entitlement: health, insurance, retirement	Personal Space	benefit_type, provider, coverage_start, premium, balance, gap_flags
Grant	Funding opportunity or awarded grant	Program	funder, amount, deadline, status, reporting_requirements
Metric	A KPI measurement or analytical data point	Any container	metric_type, value, period, target, trend
GovernanceProposal	Formal governance action requiring member vote	Organization Space	proposal_type, status, votes, quorum, decision_timestamp
CrowdresourcingCampaign	Campaign soliciting contributions from community	Community Space	goal, progress, contributors, reward_structure, deadline
MarketplaceListing	Published listing on Kogi Marketplace	Resource or Artifact	category, price, currency, visibility, sales_count, rating

15. Sheet System — 35 Derived Sheets

A Sheet is a named, scoped, materialized view of the PortfolioComponent registry. Sheets are not separate databases — they are views over the single underlying PortfolioRow store, defined

by: a SheetDefinition (filter predicates, visible columns, default sort, default grouping) and an optional ComputedColumnSet (additional engine-derived columns rendered only on this sheet).

Sheet ID	Sheet Name	Primary Row Types	Default Group By	Key Use Case
SHT-001	Master Registry	All types	Type then Status	Root view of all components; global search and bulk operations
SHT-002	Portfolio Hierarchy	Portfolio, SubPortfolio, Program, Project	Hierarchy (tree)	Visualize structural tree with rollup metrics
SHT-003	Programs	Program	Status	Strategic initiative tracking and KPI alignment
SHT-004	Projects	Project	Program → Status	Operational project management with methodology and sprint data
SHT-005	Tasks & Backlog	Task, Story, Epic, Feature	Project → Sprint	Work execution: backlog, sprint, kanban
SHT-006	Resources	Resource	Resource Type	Manage human, machine, license, and service resources
SHT-007	Assets	Asset	Asset Type	Capital assets, IP, financial assets, digital assets
SHT-008	Artifacts	Artifact	Project → Artifact Type	Versioned outputs: documents, designs, code, reports
SHT-009	Finances	All financial rows	Quarter → Category	Unified P&L: income, expenses, budget across all engagements
SHT-010	Budget Tracker	Program, Project, Gig, Contract	Program → Status	Budget allocation, spend, remaining, and utilization
SHT-011	Work & Gigs	Gig, Contract, Job, Task	Platform → Status	All income-generating engagements across all platforms
SHT-012	Deliverables	Project, Artifact, Release	Project → Status	Output tracking: what was produced and when
SHT-013	Timeline	All dated rows	Quarter → Program	Temporal view of all work with milestones
SHT-014	Roadmap	Program, Project, Milestone	Program → Quarter	Strategic roadmap for external sharing

Sheet ID	Sheet Name	Primary Row Types	Default Group By	Key Use Case
SHT-015	Portable Benefits	BenefitAccount	Benefit Type	Benefits dashboard: balances, contributions, coverage gaps
SHT-016	Grants & Microfinancing	Grant, Campaign (grant type)	Status	Grant pipeline from discovery to disbursement and reporting
SHT-017	Equity & Crowdfunding	Campaign, Investment	Campaign Type	Fundraising, equity rounds, and investment positions
SHT-018	Group Economics	Shared Portfolio, Revenue Pool, Org	Organization	Cooperative and collective financial structures
SHT-019	Organizations	Profile (org type)	Org Type	All organizations and cooperatives the user belongs to
SHT-020	Shared Portfolios	Portfolio (shared)	Owner Org	Co-owned and shared portfolio components
SHT-021	Collaboration	All shared/contributed rows	Contributor	Contribution tracking and attribution weights
SHT-022	Risk Register	All rows with risk_flags	Risk Severity	Unified risk dashboard across all components
SHT-023	Analytics Dashboard	All rows	Type → Health Score	Health scores, performance metrics, AI signals
SHT-024	Feed & Community	Profile, Post, Artifact (public)	Space	Community-facing content and social activity
SHT-025	Contacts & Network	Profile, Contact	Relationship Type	CRM: client, collaborator, investor, and vendor contacts
SHT-026	Marketplace Listings	Asset, Resource, Gig (public)	Category	Published marketplace items and their performance metrics
SHT-027	Exchange	Investment, Deal, Campaign, Asset	Exchange Type	Exchange market activity: bids, deals, instruments
SHT-028	Archive	All archived rows	Archive Date	Deep storage with full restore capability
SHT-029	Event Log	EventLog entries (read-only)	Date → Actor	Immutable audit trail of all mutations
SHT-030	Snapshots	Snapshot records	Date	Point-in-time portfolio snapshots for restore

Sheet ID	Sheet Name	Primary Row Types	Default Group By	Key Use Case
SHT-031	Link Network	LinkEdge records	Edge Type → Target	All inter-portfolio links; KLNK graph in spreadsheet form
SHT-032	Identity & Profiles	ProfilePartition records	Identity	Multi-identity management: profiles, visibility masks, accounts
SHT-033	Benefits Deep Dive	BenefitAccount + Gig (contribution)	Benefit Type → Platform	Gig platform contribution tracking and benefit projection
SHT-034	Custom Sheet (User)	User-defined	User-defined	Ad-hoc views for specific workflows or client reports
SHT-035	Template Library	ItemBook:Template rows	Category	Reusable templates from KOGI-APPSTORE and user's library

16. Kogi Hypercubes on Hypergrid

Cube Name	N	D_1	D_2	D_3	D_4	Primary Entities
kogi.portfolio.components	2	EntityAxis (ComponentId)	PropertyAxis (field name)	—	—	All PortfolioComponents: Items and Containers
kogi.portfolio.kpis	3	EntityAxis (ComponentId)	PropertyAxis (metric_name)	TimeAxis (period)	—	KPI measurements over time periods
kogi.portfolio.finances	3	EntityAxis (AccountId)	PropertyAxis (field_name)	TimeAxis (period)	—	Financial records: income, expenses, allocations
kogi.portfolio.benefits	2	EntityAxis (BenefitId)	PropertyAxis (field_name)	—	—	Benefits coverage records
kogi.portfolio.actions	2	EntityAxis (ActionId)	PropertyAxis (field_name)	—	—	ActionKind event records for activity log
kogi.portfolio.approvals	2	EntityAxis (ApprovalId)	PropertyAxis (field_name)	—	—	ApprovalRequest records for governance workflows
kogi.portfolio.links	2	EntityAxis (LinkEdgeId)	PropertyAxis (field_name)	—	—	KLNK InterPortfolioLink records
kogi.portfolio.snapshots	2	EntityAxis (SnapshotId)	PropertyAxis (field_name)	—	—	Point-in-time portfolio snapshot records

17. kogi-engine and Oba AI

17.1 kogi-engine Architecture

kogi-engine is the Scala 3 analytics and intelligence engine that consumes all portfolio events from the Kogi EventLog via Kafka and writes computed signals back into the portfolio via the WritebackService. It operates as a streaming computation engine with 20+ sub-engines, each responsible for a specific class of AI-derived columns. kogi-engine is event-driven: it subscribes to the portfolio.* Kafka topics and triggers computation on relevant mutations.

Sub-Engine	Output Attribute	Input Attributes	Trigger	Hypercube
KogiHealthScoreEngine	health_score	progress_pct, budget_utilization, schedule_variance, risk_flags, velocity	Any component mutation	kogi.portfolio.l
KogiRiskEngine	risk_score	risk_flags, critical_risk_count, overdue_milestones, budget_burn_rate	Risk record change or deadline approach	kogi.portfolio.l
KogiAlignmentEngine	alignment_score	okr_ids, kpi_targets, milestone_completion_rate	Program or strategy mutation	kogi.portfolio.l
KogiCollaborationEngine	collaboration_score	contributor_count, contribution_distribution, engagement_rate	Contribution event	kogi.portfolio.l
KogiMaturityEngine	maturity_score	artifact_count, version_history, review_completion_rate	Artifact change	kogi.portfolio.ponents
KogiAnomalyEngine	anomaly_flag	All numeric metrics (rolling baseline)	Metric deviation > 2σ from rolling baseline	kogi.portfolio.ponents
KogiMatchEngine	match_score	skills, requirements, availability, location, portfolio_quality	Skill or requirement update	kogi.portfolio.l
KogiIncomeProjectionEngine	income_projection_90d	gig_history, contract_pipeline, market_rates, platform_trends	Gig or contract change	kogi.portfolio.ces
KogiBudgetEngine	budget_burn_rate	budget_spent, budget_allocated, historical_spend_rate	Financial transaction	kogi.portfolio.ces
KogiNarrativeEngine	narrative_summary	milestone_completions, artifact_outputs, key_metrics	Significant milestone completion	kogi.portfolio.ponents
KogiScheduleRiskEngine	predicted_completion_date	velocity, remaining_work, schedule_variance, team_capacity	Sprint completion or task update	kogi.portfolio.ponents
KogiCoverageGapEngine	coverage_gap_flags	benefit_accounts, income_replacement_ratio, pto_remaining	Benefit or gig change	kogi.portfolio.fits

17.2 Oba AI — The Portfolio Chief of Staff

Oba is the Kogi platform AI assistant — the portfolio's AI chief-of-staff. It reads every row, every column, and every engine signal, and surfaces proactive, context-aware intelligence directly in the spreadsheet UI. Oba does not just answer questions; it notices what the user should notice, flags what they should act on, and prepares actions for their approval.

Oba Mode	Trigger	Examples
Reactive	User types a natural language query in the Oba bar	'Show me everything at risk this week' → executes filtered HyperQL; 'What's my income projection for Q3?' → reads KogiIncomeProjectionEngine output
Proactive	EventLog mutation triggers Oba annotation rules	'This project is 14 days behind schedule — adjust due date or reduce scope?'; 'This grant deadline is in 5 days — draft application now?'; 'Budget utilization at 87% — review spend?'
Column Completion	Required field detected as empty on a new row	'Due date is missing — suggest April 30 based on similar projects?'; 'No risk flags — would you like me to analyze and create an initial risk assessment?'
Smart Filters	User asks a question that implies a filter	'Show me what needs attention today' → ViewFilter: { risk_score > 60 OR due_date < TODAY+3 AND status = Active }
Formula Suggestions	User adds a custom FormulaColumn	'You're tracking revenue and expenses — want me to add a profit margin % formula column?'
Batch Actions	User states a bulk intent	'Archive all completed projects older than 90 days' → Oba prepares BatchArchiveAction for user confirmation
Sheet Generation	User requests a new view	'Create a sheet showing all income sources this quarter grouped by platform' → Oba generates ViewDefinition and creates SHT-034 instance
Health Briefing	Daily scheduled run	Top health risks, upcoming deadlines, pending approvals, contribution opportunities, suggested grant matches
Narrative Generation	User requests portfolio narrative for external use	'Write a portfolio highlight for my top 3 projects this quarter' → Oba generates structured narrative from row data for resumes, proposals, grant applications
Autonomous (approval)	User delegates a task to Oba	Oba executes only after explicit user approval of a prepared action plan. No autonomous execution without confirmation.

18. KLNK — The Kogi Link Network

18.1 Overview

KLNK is the inter-portfolio graph system that maps the full network of connections between Kogi users, portfolios, organizations, and federation nodes. Every connection between workers, between a worker and an organization, between a portfolio and a marketplace listing, between a gig worker and a client — is a typed LinkEdge in the KLNK graph. The full graph of all LinkEdges and nodes is the Kogi economic graph — the living map of the platform's collaborative and economic relationships.

18.2 LinkEdge Types

LinkEdgeType	Direction	Consent	Description	Creates ShadowRow?
Collaborates	Undirected	Yes — mutual	Active collaboration on a shared component	Yes — in both parties' spreadsheets
InvestedIn	Directed (A→B)	Yes — from B	Economic investment or patronage	Yes — investor sees investee's approved columns
Employs	Directed (org→w)	Yes — from w	Organization employs or contracts a worker	Yes — org sees worker's work record columns
Contracted	Directed (A→B)	Yes — from B	A formal contract between two parties	Yes — both parties see agreed delivery columns
Follows	Directed (A→B)	No	A subscribes to B's public updates	No — read-only via public visibility
Endorses	Directed (A→B)	No	A endorses a skill or work product of B	No
SharedPortfolio	Directed (own→)	Yes — from ctb	Contributor has access to a shared portfolio	Yes — contributor sees permitted columns of shared portfolio
MarketplaceTransaction	Directed (buy→)	Via transaction	Completed marketplace transaction	Yes — both parties see transaction record
OrgMembership	Directed (org→m)	Yes — from m	Member belongs to an organization or coop	Yes — member sees org's public shared portfolio
DependsOn	Directed (A→B)	No	A's work depends on B's output	Yes — A sees B's delivery status columns
ResourceShares	Directed (own→)	Yes — from rcvr	A shares a resource (tool, template) with B	Yes — receiver sees resource access columns
FederationPeer	Undirected	Grid-level trust	Two PortfolioSystems in a federation	Yes — via full CrdtLog sync across agreed cubes

18.3 LinkForest Visualization Modes

View Mode	Description	Depth	Key Use Case
Link Forest View	All LinkTrees rooted at all user identities, rendered side-by-side as multi-root forest	3	Full connectivity overview; identify isolated components; find bridge nodes
Component Tree View	Single LinkTree rooted at a specific portfolio component, showing all inbound and outbound links	5	Understand a component's full network context; due diligence for deals
Dependency Tree View	Filtered tree showing only DependsOn edges, rendered as	Unlimited	Cross-portfolio critical path analysis; dependency risk assessment

View Mode	Description	Depth	Key Use Case
	dependency diagram with critical path		
Investment Graph	Filtered graph showing InvestedIn and Contracted edges — the financial network of the portfolio	3	Investor portfolio visualization; deal network analysis
Organization Tree	Org chart-style tree showing OrgMembership edges from a root organization down through members	4	Org structure visualization; governance participation overview
Collaboration Web	Bipartite graph showing Collaborates edges between users and shared portfolio components	3	Team composition analysis; contribution attribution visualization
Federation Graph	System-level graph showing FederationPeer edges between PortfolioSystem nodes	Full	Platform infrastructure; CRDT sync health monitoring
Spreadsheet Grid View	KLNK link graph as spreadsheet (SHT-031): each row = LinkEdge; columns = from/to/type/weight	N/A	Bulk link management; data export; programmatic access

19. Kogi Multi-Identity System (KPID)

19.1 Identity Hierarchy

A Sovereign Entity is the real-world person, organization, or collective that owns a root PortfolioSystem instance. A Sovereign Entity can have multiple Identities (multiple @handles or organizational identities), each with its own ProfilePartition of the root spreadsheet. Identities share the same underlying PortfolioComponent rows but have configurable visibility masks and data partitioning policies.

Identity Layer	Description	Hypergrid Mechanism
SovereignEntity	The real-world owner. Has full ownership of all data across all their identities.	Root Grid owner; PermissionTier::Owner across all cubes
Identity (Handle)	A named public-facing @handle or organizational identity. Has its own profile partition.	ProfilePartition: rows tagged with identity_id; VisibilityMask per identity
Profile	A contextual presentation of an Identity for a specific audience (public, professional).	HypercubeView DimSlice filter on profile_type attribute
Account	A login credential set associated with an Identity.	Authentication record; linked to Identity via secure cryptographic token

19.2 VisibilityMask

Observer Type	Visible Rows	Visible Columns
Public	Rows with visibility=Public	Only columns with AttrVisibility=Public
Follower	Public rows + rows with visibility=Follower	Public columns + Follower-visible columns (e.g. recent activity feed)
Connection	Follower rows + rows with visibility=Connection	Follower columns + Connection-visible (e.g. contact details, collaboration history)
Member	Connection rows + Space-scoped rows in shared Spaces	Connection columns + Space-visible (e.g. internal project details, task list)
Editor	Member rows + all non-private rows	Member columns + editor-level (e.g. budget, configuration, risk flags)
Admin	All rows in managed Space	All columns including governance records, full audit trail
Owner	All rows including private and system	All columns including encrypted, system, AI-write-only attributes

Part IV — Ume: Business Operating System

20. Ume Platform Overview

20.1 The Organization as Root Domain

Ume's root domain concept is the Organization. Just as the Portfolio is the center of gravity for Kogi, the Organization is the center of gravity for Ume. Every entity in an organization's world — module, employee, legal entity, marketing campaign, financial account, risk record, OKR, procurement order — is a row in the Organization System. The Organization System (Ume OS) is the master spreadsheet of the organization's entire operational world.

> *The Organization is not a container in Ume — it is the operating system. Every module, every domain, every employee, every ledger entry, every governance decision is expressed as a view over the Organization's root hyperspreadsheet. The Ume kernel supervises 42 domain modules, each of which maps to one or more Hypercubes in the Organization Grid.*

20.2 Kernel-to-Hypergrid Mapping

Ume's kernel services map directly onto Hypergrid infrastructure, replacing traditional in-memory or per-service data stores with shared Hypercubes:

Ume Kernel Service	Hypergrid Equivalent	Notes
ModuleRegistry	ume.kernel.modules Hypercube	Module registrations as HyperRows; versioned, CRDT-synced
InMemoryEventBus	Hypergrid EventLog + CrdtLog	Persistent, time-travelable event stream; dual-write architecture
RbacEngine	Hypergrid PermissionTier + AttrVisibility	Per-attribute, per-module RBAC; enforced at bridge layer
SupervisorEngine	Hypergrid GovernanceEngine + PolicyEngine	Module restart/backoff policy; supervision graph in Hypergraph
OrchestratorEngine	HyperQL + computed attributes	Workflow query and execution; orchestration state in process cubes
TemplateEngine	ume.templates.library Hypercube	Versioned template storage; 39+ organization templates
SchemaRegistry	Hypergrid AttributeKeyRegistry	Per-Hypercube schema contracts; type-safe attribute key registration
MetricsCollector	ume.analytics.kpis (D_3=TimeAxis)	Time-series metric storage; DimFold aggregation for dashboards

Ume Kernel Service	Hypergrid Equivalent	Notes
LogAuditManager	Hypergrid EventLog	Immutable, append-only audit trail; AS_OF time-travel for compliance
StorageManager	Hypergrid UniversalCellStore	All entity storage; partitioned by module cube; CRDT-consistent
SearchService	HyperQL + full-text attribute index	Cross-cube entity search; NL-to-HyperQL for user-facing queries
BackupManager	Hypergrid Snapshot + EventLog replay	Point-in-time state capture; full restore from EventLog

21. ume.kernel.modules — Module Registry Hypercube

D_2 Key	TypedAttrValue	CRDT	PermissionTier	Notes
module_id	Text	LWW	System	Canonical module identifier
name	Text	LWW	Admin	Display name (e.g. 'Finance & Accounting')
domain_area	Json	LWW	Admin	DomainArea enum value
version	Text	MaxRegister	System	Module version string — always the maximum seen
lifecycle_state	Json	Lattice	System	Registered → Starting → Running → Degraded → Stopped
status	Json	LWW	Admin	ModuleStatus: Active | Suspended | Disabled
dependencies	Json	OR-Set	Admin	Vec<ModuleId> that this module depends on
permissions_req	Json	LWW	Admin	Vec<Permission> declared by the module
executor_id	Text	LWW	System	Assigned ExecutorPool thread pool ID
restart_count	Number	GrowOnlyCounter	System	Supervisor restart counter — only increases
last_error	Json	LWW	Admin	Option<KernelError> from most recent failure
health_score	Number	LWW (AI-write)	System	AI-computed module health score 0–100
evaluation_cache	Json	LWW	System	Cached evaluation output from last evaluation run
config	Json	LWW	Admin	Module configuration blob

D_2 Key	TypedAttrValue	CRDT	PermissionTier	Notes
metrics	Json	LWW	System	MetricPoint observations snapshot

22. The 42 Organization Module Hypercubes

#	Module Name	Primary Hypercube	Key Dimensions	Key D_2 Fields
01	Organization Administration	ume.admin.org_units	D_3=TimeAxis	name, type, parent_unit, head_count, budget, head_of_unit
02	Organization Analytics	ume.analytics.kpis	D_3=TimeAxis, D_4=DomainAxis	metric_name, value, target, trend, period, owner
03	Backup & Recovery	ume.backup.jobs	D_3=CategoryAxis	job_id, scope, status, schedule, last_run, recovery_point
04	Board Management	ume.board.meetings	D_3=TimeAxis	title, date, quorum, attendees, resolution_count, status
05	Business Development	ume.bizdev.opportunities	D_3=CategoryAxis	title, stage, value, probability, close_date, owner
06	Enterprise CMS	ume.cms.content	D_3=TimeAxis	title, content_type, status, author, tags, publish_date
07	Communications	ume.comms.channels	D_3=CategoryAxis	channel_name, type, members, status, retention_policy
08	CRM	ume.crm.contacts	D_3=CategoryAxis	name, type, stage, owner, last_contact, value, source
09	Design System	ume.design.tokens	D_3=TimeAxis	token_name, value, category, version, approved_by
10	Engineering & Technology	ume.eng.projects	D_3=TimeAxis	name, stack, status, team, velocity, sprint_count
11	ESG / CSR / Sustainability	ume.esg.metrics	D_3=TimeAxis, D_4=CategoryAxis	metric_name, value, category, period, verified_by
12	Enterprise Engineering Admin	ume.enterprise.systems	—	system_name, type, status, owner, sla_target

#	Module Name	Primary Hypercube	Key Dimensions	Key D_2 Fields
13	Legal Entity (Chombo)	ume.chombo.entities	D_3=CategoryAxis (jurisdiction)	entity_name, jurisdiction, type, status, reg_number
14	Finance & Accounting	ume.finance.accounts	D_3=TimeAxis	account_name, type, balance, currency, period_close
15	GRC	ume.grc.risks	D_3=TimeAxis	title, category, severity, probability, status, owner
16	Human Resources	ume.hr.employees	D_3=TimeAxis	name, role, department, status, hire_date, salary
17	Investment Management	ume.investment.portfolio	D_3=TimeAxis, D_4=CategoryAxis	asset_name, type, value, allocation_pct, return_pct
18	IT & Asset Management	ume.it.assets	D_3=CategoryAxis	asset_name, type, status, owner, cost, depreciation
19	Enterprise Knowledge	ume.knowledge.articles	D_3=TimeAxis	title, category, status, author, views, last_updated
20	Learning & Development	ume.learning.programs	D_3=TimeAxis	title, type, status, enrolled, completed, completion_rate
21	Management & Strategy	ume.strategy.okrs	D_3=TimeAxis	objective, key_result, progress, owner, quarter
22	Marketing (Soko)	ume.soko.campaigns	D_3=TimeAxis, D_4=CategoryAxis	name, type, status, budget, ctr, conversion_rate
23	Master Data Management	ume.mdm.entities	D_3=CategoryAxis	entity_name, type, golden_record_id, source, quality_score
24	Office & Facility	ume.facilities.sites	D_3=CategoryAxis	site_name, type, capacity, status, cost_per_month
25	Operations Management	ume.ops.processes	D_3=TimeAxis	name, type, status, sla_target, sla_actual, owner
26	Portal / Hub / Dashboard	ume.portal.dashboards	—	title, owner, widgets, layout, visibility, audience
27	Portfolio & Program	ume.portfolio.programs	D_3=TimeAxis	name, status, budget, kpi_count, health, sponsor

#	Module Name	Primary Hypercube	Key Dimensions	Key D_2 Fields
28	PR & Branding	ume.branding.assets	D_3=TimeAxis	name, type, status, campaign_id, version, usage_rights
29	Process, Orchestration & Workflow	ume.process.workflows	D_3=TimeAxis	name, type, status, trigger, step_count, avg_duration_ms
30	Product, Services & Solutions	ume.product.catalog	D_3=TimeAxis, D_4=CategoryAxis	name, type, status, sku, pricing_model, revenue_ytd
31	Production, Manufacturing	ume.production.batches	D_3=TimeAxis	batch_id, product, status, quantity, date, defect_rate
32	Requirements Management	ume.requirements.specs	D_3=CategoryAxis	title, type, priority, status, owner, linked_solution_id
33	Enterprise Risk Management	ume.risk.register	D_3=TimeAxis	title, category, severity, probability, mitigation_status
34	Sales Management	ume.sales.opportunities	D_3=TimeAxis	title, stage, value, owner, close_date, win_probability
35	Enterprise Schedule	ume.schedule.events	D_3=TimeAxis	title, type, start_at, end_at, attendees, location
36	Security, Privacy & Protection	ume.security.incidents	D_3=TimeAxis	title, severity, status, assignee, cve_id, resolution_date
37	Logistics, Supply Chain & WMS	ume.supply_chain.orders	D_3=TimeAxis	order_id, supplier, status, total, eta, tracking_id
38	Team & Cooperative Management	ume.teams.groups	—	name, type, members, owner, mission, governance_model
39	Organization Templating	ume.templates.library	D_3=TimeAxis	name, domain, version, status, uses_count, author
40	Enterprise Work Management	ume.work.items	D_3=TimeAxis	title, type, priority, status, assignee, sprint_id
41	Custom UME Modules	ume.custom.{vendor}.*	Vendor-defined	Platform-extensible module namespace
42	Custom Org Modules	ume.org.{slug}.*	Org-defined	Organization-extensible module namespace

23. Chombo (Legal Entity) and Soko (Marketing) on Hypergrid

23.1 Chombo — Jurisdictional N=3 Hypercube

Chombo is Ume's legal entity management subsystem, covering 45 policy packs across different legal jurisdictions and compliance frameworks. It uses a 3-dimensional Hypercube so that a single multinational legal entity can have jurisdiction-sliced data in the same Hypercube:

```
ume.chombo.entities (N=3)
D₁ = EntityAxis (Uuid)            ← LegalEntityId
D₂ = PropertyAxis (String)        ← entity field name
D₃ = CategoryAxis (String)        ← jurisdiction ("US-DE", "UK", "EU", "ZA", ...)

Example cells:
cell[(entity_id, "compliance_score", "US-DE")].value = Number(94.2)
cell[(entity_id, "open_findings",    "UK")].value    = Number(3)
cell[(entity_id, "last_evaluated",   "EU")].value    =
DateTime("2026-03-15T14:30:00Z")

Cross-jurisdictional compliance report:
SELECT D₃.jurisdiction, AVG(cell[D₁, "compliance_score", D₃].value) AS
avg_compliance
FROM ume.chombo.entities
WHERE D₃.jurisdiction IN ("US-DE", "UK", "EU")
GROUP BY D₃.jurisdiction;
```

23.2 Soko — Multi-Segment Campaign Analytics N=4 Hypercube

Soko, Ume's marketing system, manages campaigns, signals, and 70 strategy packs. Its 4-dimensional Hypercube supports multi-segment, multi-period analytics natively:

```
ume.soko.campaigns (N=4)
D₁ = EntityAxis (Uuid)            ← CampaignId
D₂ = PropertyAxis (String)        ← metric/field name
D₃ = TimeAxis (Timestamp)         ← time period
D₄ = CategoryAxis (String)        ← audience_segment

Single point lookup — CTR for Campaign X, enterprise segment, Q2 2026:
cell[(campaign_id, "ctr", "2026-Q2", "enterprise")].value = Number(0.0342)

DimExpand pivot — all segments for a campaign in Q2 2026:
SELECT D₁.campaign_id,
       EXPAND D₄ AS COLUMNS(AVG(cell.value))
FROM ume.soko.campaigns
WHERE D₂.metric_name = 'ctr' AND D₃.period = '2026-Q2';
-- Result: campaign_id | enterprise_ctr | smb_ctr | consumer_ctr | ...
```

Part V — Qala: Solution Factory Operating System

24. Qala Platform Overview

24.1 The Solution as Root Domain

Qala's root domain concept is the Solution. The Solution is the center of gravity for everything in Qala: every SDE, every artifact, every CCR, every release, every toolchain, every AI signal — all exist to produce, govern, and distribute Solutions. The Solution System (Qala OS) is the master spreadsheet of the solution factory's entire production world.

But Qala goes further: the Solution is not just a thing that is produced by a factory — it is the universal category. A Solution Factory is itself a Solution. Kogi (the Independent Worker OS) is a Solution produced by the Qala Root Factory. Ume (the Business OS) is a Solution produced by the Qala Root Factory. Every product, service, platform, formulation, instrument, dataset, creative work, or software system in the Apapo ecosystem is a Solution in Qala's ontology.

> *The Root Factory Principle: Qala itself is a Solution Factory. The Qala platform governs itself through its own governance framework. Every Kogi and Ume platform release goes through a CCR in the Qala Root Factory. The Qala platform's own SDE is a first-class HyperRow in qala.sdes. This self-describing architecture gives the operations team the full power of HyperQL, AI signals, and governance workflows to manage the platform itself.*

24.2 Qala's Six-Plane Architecture

Plane	Language	Hypergrid Mapping	Responsibility
Kernel Plane	Rust	hypergrid::core::Grid — integrity, service registry, event aggregation, solution graph engine	Platform integrity; service lifecycle; event aggregation across all cubes
Control Plane	Go	Domain-layer systems on Hypergrid: SDE, Factory, Identity, CCR, Release services	Factory and SDE lifecycle; CCR workflow; release pipeline; identity management
Execution Plane	Go + Scala	HyperQL query engine + computed attribute execution; CI/CD pipeline state in qala.cicd.* cubes	Build and test execution; pipeline orchestration; deployment state tracking
Intelligence Plane	Rust	HG-AI + AttrComputation::AiEngine for all AI agents; SEM writes to qala.security.* cubes	AI Agent: 10 capabilities across SDE and Solution cubes; security threat analysis

Plane	Language	Hypergrid Mapping	Responsibility
Platform Operations	Go + Terraform	HG-OPS: Snapshot, Federation, deployment topology management	Infrastructure management; federation health; disaster recovery; telemetry
Domain Extension Plane	Go + YAML DSL	HypercubePlugin trait — Domain Packs as plugins registering new axis types, attributes, computed models	Domain Pack ecosystem; regulatory extension; industry-specific governance and compliance

25. Qala Hypercube Inventory

Cube Name	N	Dimensions	Primary Entities	Key D_2 Fields
qala.factories	2	Entity, Property	Solution Factory records	factory_id, name, domain_packs, governance_config, status, root_factory_flag, owner_org_id
qala.sdes	3	Entity, Property, Time	Solution Development Environment records (versioned)	sde_id, factory_id, name, status, team_members, toolchain_id, hermetic_manifest, drift_status
qala.solutions	2	Entity, Property	Solution records (all types and versions)	solution_id, sde_id, type, version, lifecycle_state, quality_score, defect_density, is_factory
qala.solutions.metrics	4	Entity, Metric, Time, Category	Solution quality metrics multi-dim	quality_score, defect_density, compliance_score, security_score, performance_score, regulatory_risk
qala.ccrs	2	Entity, Property	Change Control Request records	ccr_id, solution_id, change_type, risk_level, status, approvers, decision_timestamp, rationale
qala.releases	2	Entity, Property	Release records	release_id, solution_id, version, release_type, distribution_channels, release_notes, signed_by
qala.artifacts	2	Entity, Property	Artifact records: build, test, compliance, security	artifact_id, solution_id, artifact_type, format, hash, created_at, signed_by, retention_policy
qala.toolchains	2	Entity, Property	Toolchain configuration records	toolchain_id, name, tools, versions, environment_type, hermetic_digest, validated_by
qala.ai_recommendations	2	Entity, Property	AI Agent recommendation records	rec_id, solution_id, recommendation_type,

Cube Name	N	Dimensions	Primary Entities	Key D_2 Fields
				confidence, action_proposed, status, accepted_at
qala.cicd.pipelines	3	Entity, Property, Time	CI/CD pipeline execution records	pipeline_id, sde_id, trigger, status, duration_ms, test_pass_rate, deploy_target
qala.security.events	3	Entity, Property, Time	Security incident and threat records	event_id, solution_id, threat_type, severity, status, cve_id, remediation_status
qala.governance.records	2	Entity, Property	Governance pack evaluation records	record_id, solution_id, pack_id, evaluation_result, open_findings, last_evaluated

26. The SDE Versioned Hypercube

The SDE (Solution Development Environment) uses a 3-dimensional Hypercube (Entity × Property × Time) to capture the full version history of every SDE configuration. This enables hermetic build integrity: every historical state of the SDE — its toolchain, environment manifest, team composition, dependency pins — is addressable by timestamp.

```
qala.sdes (N=3)
D₁ = EntityAxis (Uuid)           ← SdeId
D₂ = PropertyAxis (String)       ← field name
D₃ = TimeAxis (Timestamp)        ← snapshot timestamp

Reading current SDE state:
cell[(sde_id, "toolchain_ids",     NOW)].value = Json([...])
cell[(sde_id, "hermetic_manifest", NOW)].value = Json({...})
cell[(sde_id, "drift_status",      NOW)].value = Text("Clean")

Reading SDE state at a historical point in time (AS_OF):
SELECT *
FROM qala.sdes
AS_OF '2026-01-15T10:00:00Z'
WHERE D₁.id = '{sde_id}';

Rollback operation (atomic cross-D₂ write to D₃=now):
-- Read all D₂ keys at D₃=target_timestamp
-- Write those values to D₃=NOW as a single atomic transaction
-- EventLog records the rollback as a RollbackEvent audit entry

Drift detection (DriftDetectionEngine):
-- Subscribes to EventLog mutations on environment_manifest, toolchain_ids,
hermetic_manifest
-- On mutation: hash(current_sde_state) vs hash(pinned_hermetic_manifest)
-- If hash differs: drift_status := Drifted; alert published to Operations channel
-- Target detection latency: < 15 minutes from drift event
```

27. Solution Lifecycle as Hypergrid Lattice

The Solution lifecycle in Qala is enforced at the CRDT level using a Lattice CRDT on the lifecycle_state attribute key. This ensures that concurrent state-transition operations from multiple federation nodes never result in an invalid lifecycle state — no matter how many nodes simultaneously attempt transitions, the lattice merge always resolves to a valid forward state.

```
// Qala solution lifecycle lattice — partial order defining valid forward
transitions
let lifecycle_lattice = LatticeOrder::new(vec![
    LatticeNode { state: "Draft",      successors: vec!["InReview"] },
    LatticeNode { state: "InReview",   successors: vec!["Approved", "Draft"] },
    LatticeNode { state: "Approved",   successors: vec!["Active", "InReview"] },
    LatticeNode { state: "Active",     successors: vec!["Deprecated"] },
    LatticeNode { state: "Deprecated", successors: vec!["Retired"] },
    LatticeNode { state: "Retired",    successors: vec![] },  // terminal
]);

// If concurrent transitions occur: Draft→InReview (node A) and InReview→Draft
(node B)
// Lattice merge: join("Draft", "InReview") = "InReview" (forward state wins)
// CRDT guarantee: lifecycle_state never regresses without explicit Admin override
```

Lifecycle State	Description	Entry Gate	Exit Conditions
Draft	Initial creation; work in progress; no governance constraints	None	Developer submits for review → InReview
InReview	Formal review process initiated; reviewers assigned via CCR	At least one CCR created	All required approvers sign → Approved; any approver rejects → Draft
Approved	All required approvals obtained; staged for release	All CCR approvers have signed	Staging validation passes → Active; staging fails → InReview
Active	Publicly active and distributed; consumers can adopt	Release governance pack sign-off	Deprecation decision → Deprecated
Deprecated	End-of-life announced; existing consumers notified; no new adoption	Deprecation notice published	Deprecation notice period elapsed → Retired
Retired	Permanently retired; read-only historical record; EventLog sealed	Deprecation period complete	Terminal state — no further transitions

28. Qala AI Agent — Ten Capabilities

AI Capability	Plugin	Output Attribute	Target Cube	Update Trigger
SDE Optimization	SdeOptimizationEngine	optimization_recommendations	qala.sdes	On SDE team or toolchain change
Pipeline Bottleneck Detection	PipelineAnalysisEngine	bottleneck_report	qala.cicd.pipelines	On pipeline run completion
Predictive Defect Detection	DefectPredictionEngine	defect_risk_score	qala.solutions	On code metrics or test coverage change
Test Case Generation	TestGenEngine	suggested_test_cases	qala.solutions	On requirement or specification change
Anomaly Detection	AnomalyEngine	anomaly_flags	qala.solutions.metrics	On metric deviation > 2σ
Resource Prediction	ResourceForecastEngine	forecast_compute_units	qala.sdes	On sprint planning or team change
Content / Doc Suggestions	ContentSuggestionEngine	documentation_suggestions	qala.solutions	On significant code or spec change
Security Threat Analysis	SecurityThreatEngine	threat_report	qala.security.events	On CVE feed update or dependency change
Backup Schedule Optimization	BackupOptimizationEngine	optimal_backup_schedule	qala.sdes	On SDE data volume or access pattern change
Prototype Feedback Analysis	PrototypeFeedbackEngine	feedback_synthesis	qala.solutions	On new user feedback or test results submission

29. Domain Packs

A Domain Pack is a curated bundle of HypercubePlugin extensions that provides complete domain-specific governance, compliance, AI, and attribute schemas for solutions in a particular regulated industry. Domain Packs are installed at the factory level and can be composed — a solution can be governed by multiple packs simultaneously.

Domain Pack	Industry	Key Capabilities	Regulatory Scope
Software (Default)	Software / Technology	Code metrics, test coverage, SAST/DAST, dependency audit, deployment validation, security posture scoring	SOC 2, ISO 27001, OWASP Top 10
Pharmaceutical	Pharmaceutical	Batch formula versioning, GMP compliance, ingredient specs, stability data, regulatory submissions, regulatory_risk AI model	FDA 21 CFR 211, EMA, ICH Q10, GxP
Financial Instrument	Financial Services	KYC/AML module, risk model validation, regulatory capital computation, stress test orchestration, market risk scoring	SEC, FCA, FINRA, Basel III/IV
Medical Device	Medical Device	Design history file, ISO 14971 risk management, V&V tracking, adverse event monitoring, post-market surveillance	FDA 21 CFR 820, ISO 13485, MDR
Research Dataset	Academia / Research	Provenance tracking, IRB approval, data anonymization, citation graph, reproducibility score, FAIR data compliance	IRB, GDPR, NIH, FAIR principles
Creative Works	Creative / Media	IP ownership verification, license management, attribution chain, similarity detection (copyright), rights clearance	Copyright offices, CMOs, Creative Commons
Government / Public	Government	FedRAMP controls, FISMA compliance, authority-to-operate workflow, security categorization, data classification	FedRAMP, FISMA, NIST SP 800-53
Aerospace / Defense	Aerospace & Defense	DO-178C certification, ITAR compliance, safety case management, failure mode analysis, airworthiness records	FAA, EASA, DO-178C, MIL-STD

29.1 Domain Pack Plugin Example — Pharmaceutical

```
pub struct PharmaDomainPack;

impl HypercubePlugin for PharmaDomainPack {
    fn plugin_id(&self) -> &str { "qala.domain_pack.pharma.v2" }
    fn name(&self) -> &str { "Pharmaceutical Formulation Domain Pack" }
```

```
    fn version(&self) -> &str { "2.0.0" }

    fn attribute_key_defs(&self) -> Vec<AttributeKeyDef> {
        vec![
            lww_text("batch_formula_version", "Batch Formula Version"),
            lww_text("pharmacopoeial_grade",   "Pharmacopoeial Grade"),
            lww_json("ingredient_specs",       "Ingredient Specifications"),
            lww_json("regulatory_submissions", "Regulatory Submissions"),
            lww_text("gmp_compliance_status",  "GMP Compliance Status"),
            lww_json("stability_data",         "Stability Study Data"),
            // AI-computed regulatory risk (system-write only)
            AttributeKeyDef {
                key: "regulatory_risk_score".into(),
                attr_type: AttributeType::Ai,
                write_permission: PermissionTier::System,
                computation: Some(AttrComputation::AiEngine {
                    plugin_id: "qala.pharma.regulatory_risk_engine",
                    model_name: "regulatory_risk_v2".into(),
                    parameters: HashMap::new(),
                }),
                crdt_semantics: CrdtSemantics::LastWriteWins,
                ..Default::default()
            },
        ]
    }

    fn ai_engine(&self) -> Option<Box<dyn AIEngineAdapter>> {
        Some(Box::new(PharmaRegulatoryRiskEngine::new()))
    }
}
```

Part VI — Cross-Platform Integration, Federation & Deployment

30. Cross-System Data Gravity

Because Kogi, Ume, and Qala all store their entities in the same Universal Cell Store (or in federated UCS instances that sync via CrdtLog), cross-system queries and integrations are first-class HyperQL operations rather than engineering challenges requiring ETL pipelines, data warehouses, or synchronization jobs.

Cross-System Integration	HyperQL Pattern	Consent Required
Kogi portfolio item linked to Qala solution	JOIN kogi.portfolio.components TO qala.solutions ON GRAPH_EDGE('CrossGridLink')	Yes — worker grants factory access to portfolio item
Ume employee linked to Kogi portfolio	JOIN ume.hr.employees TO kogi.portfolio.components ON GRAPH_EDGE('Employs')	Yes — worker grants org access to work record columns
Qala solution attributed to Kogi worker	TRAVERSE GRAPH FROM qala.solutions EDGE_TYPE=Produces DIRECTION=Inbound	Via Collaborates or InvestedIn link establishment
Ume org OKR cascading to Qala SDE goals	JOIN ume.strategy.okrs TO qala.sdes ON GRAPH_EDGE('Association')	No — organizational relationship within federated deployment
Kogi coop members and their Qala contributions	TRAVERSE kogi.portfolio.components EDGE_TYPE=OrgMembership; JOIN qala.solutions ON CrossGridLink	Yes — per-worker consent
Qala solution quality in Ume product catalog	JOIN ume.product.catalog TO qala.solutions ON GRAPH_EDGE('Produces')	No — Ume org owns both entities in federated deployment
Ume anomaly detection correlated with Qala drift	JOIN ume.kernel.modules TO qala.sdes ON GRAPH_EDGE('Association') WHERE drift_status!='Clean'	No — same federated deployment

31. Deployment Topology

31.1 Single-Node Development Deployment

```
[Domain System Binary (Kogi | Ume | Qala)]
  └── Grid (single node)
        ├── Hypercube(s)
        ├── UniversalCellStore (in-memory or SQLite)
        ├── Hypergraph (in-memory)
        ├── EventLog (in-memory; optional SQLite flush)
        └── CrdtLog (no federation peers)

Use cases: local development, kogi-local desktop app, Qala personal factory,
           solo Ume instance, offline-capable mobile client
```

31.2 Federated Production Deployment

```
[Load Balancer / API Gateway]
    ├── [Domain System Node 1]
    │     └── Grid (node-1)
    │           ├── UniversalCellStore (PostgreSQL-backed, sharded by grid_id)
    │           ├── CrdtLog → Kafka topic: crdt.delta.{grid_id}
    │           ├── EventLog → Kafka topic: events.audit.{grid_id}
    │           └── AI compute requests → Kafka: ai.compute.{grid_id}
    │
    ├── [Domain System Node 2..N]  (same structure, up to 255 peers)
    │
    └── [Shared Infrastructure]
          ├── Apache Kafka (crdt.delta.* · events.audit.* · ai.compute.*)
          ├── PostgreSQL (primary cell store; read replicas per region)
          ├── Redis (CrdtLog hot-path buffer; Workspace session cache)
          ├── ClickHouse (analytics: N>4 cubes; EventLog data lake)
          └── kogi-engine / Ume AI / Qala AI Agent (Kafka consumers + gRPC
WritebackService)
```

31.3 Cross-System Integrated Ecosystem Deployment

```
[Kogi Grid]  ←──── CrossGridLink ────→  [Qala Grid]
      │                                       │
      └──────── CrossGridLink ──────→  [Ume Grid] ──────┘

[Federation Coordinator Service]
  ├── CrossGridLink consent flow management
  ├── Shadow cell sync routing (Kafka-mediated)
  ├── Cross-grid HyperQL fan-out query coordination
  └── Federation health monitoring

Data residency enforcement:
  ├── GeoAxis on TenantAxis → cells stored only in approved regions
  ├── GDPR adequacy region routing via federation node selection
  └── Per-Grid data sovereignty configuration
```

32. Persistence Architecture

Storage Layer	Backend	Use Case	Notes
Primary cell store	PostgreSQL 16+	OLTP cell reads/writes for N≤4 cubes	Row-level security at DB layer. BRIN indexes for TimeAxis. GIN for JSONB. Partitioned by grid_id.
Analytics backend	ClickHouse	Analytics for N>4 cubes and high-throughput DimFold/DimExpand	Receives Kafka delta stream from primary PG. Used when result sets >1M rows or aggregation complexity >O(n²).
CRDT hot buffer	Redis	CrdtLog in-flight operation buffer; Workspace session state cache	Volatile; durable CrdtLog in PostgreSQL. Redis is the hot-path write buffer only. TTL: 24h for pending ops.
EventLog archive	S3 / Object Store	EventLog cold archive beyond 10K-event hot cap; Parquet exports; AI model artifacts	EventLog entries flushed to S3 after hot cap. Restorable on demand for time-travel beyond hot window.
Graph backend	PostgreSQL (v1); Neo4j (v2.5+)	Hypergraph edge storage and traversal	PostgreSQL recursive CTEs for graphs <1M nodes. Neo4j for KLNK at platform scale (>1M nodes).
Local / Edge store	SQLite	Single-user offline deployment; edge device client	Used for kogi-local desktop and offline-first mobile clients. Sync to PostgreSQL on reconnect.

32.1 Physical Cell Store Schema

```
-- Primary cell store (PostgreSQL)
CREATE TABLE hypergrid_cells (
    grid_id         UUID NOT NULL,
    cube_id         UUID NOT NULL,
    dim1_key        UUID NOT NULL,              -- EntityAxis key
    dim2_key        TEXT NOT NULL,              -- PropertyAxis key (field name)
    dim3_key        TEXT,                       -- Optional: TimeAxis /
CategoryAxis
    dim4_key        TEXT,                       -- Optional: GeoAxis / TenantAxis
    -- dim5_key..dim16_key for higher-dim cubes (sparse: NULL if unused)
    attr_value      JSONB NOT NULL DEFAULT '{}', -- TypedAttrValue as JSONB
    attr_version    BIGINT NOT NULL DEFAULT 0,
    attr_crdt_ts    BIGINT NOT NULL,            -- Logical clock timestamp (for
LWW)
    last_actor      TEXT NOT NULL,              -- node_id:identity_tag
    last_mutated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    PRIMARY KEY (grid_id, cube_id, dim1_key, dim2_key, dim3_key, dim4_key)
) PARTITION BY HASH(grid_id);                   -- Horizontal sharding by grid

-- EventLog (append-only, never updated or deleted)
CREATE TABLE hypergrid_events (
    event_id        UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    grid_id         UUID NOT NULL,
    cube_id         UUID NOT NULL,
    dim_keys        JSONB NOT NULL,             -- Full N-dim coordinate tuple
```

```
    attr_key        TEXT NOT NULL,
    before_value    JSONB,
    after_value     JSONB NOT NULL,
    crdt_op         TEXT NOT NULL,              -- LWW | OR-Set-Add | OR-Set-Remove
| Counter | Lattice
    vector_clock    JSONB NOT NULL,             -- {node_id: logical_timestamp}
    actor           TEXT NOT NULL,
    timestamp       TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    CONSTRAINT no_update CHECK (true)           -- Enforced at application layer
) PARTITION BY RANGE(timestamp);               -- Partition by time for BRIN
efficiency
```

33. Performance Targets

Operation	Target p50	Target p99	Scale Bound
Single HyperCell read	< 5ms	< 20ms	Primary key lookup on dim_keys tuple; scales linearly with N
HyperRow read (all fields for one entity)	< 10ms	< 40ms	Batched read of all D_2 keys for one D_1 entity; O(field_count)
DimSlice query (N=2, < 1K rows)	< 50ms	< 200ms	PostgreSQL GIN/BTree index; scales with predicate selectivity
DimFold aggregation (N=3, < 100K rows)	< 500ms	< 2s	PostgreSQL window functions; ClickHouse for larger datasets
Graph traversal (depth=5, fan-out=10)	< 200ms	< 800ms	PostgreSQL recursive CTE for <1M nodes; Neo4j for larger graphs
AS_OF time-travel (single HyperRow)	< 100ms	< 500ms	EventLog replay for a single HyperRow; hot window in PostgreSQL
CRDT merge (single attribute, in-memory)	< 1ms	< 10ms	In-memory merge; async persistence to PostgreSQL and Kafka
AI attribute computation (Tier 2)	100ms–30s	< 60s	Async; result written via WritebackService after computation completes
CrossGridLink delta sync	< 1s	< 5s	Kafka delta propagation + ShadowCell update; target near-real-time
EventLog flush to S3 archive	< 30s	< 2min	Batch flush when hot EventLog reaches 10K-event cap per entity

Part VII — API Design and Service Architecture

34. Service Architecture

Service	Protocol	Language	Responsibility
api-gateway	HTTP/2 + gRPC-Web + WSS	Go	Auth (JWT/API key), rate limiting, routing, CORS, API versioning, HTTPS termination
portfolio-service	gRPC + REST	Go	Kogi KPMS/KPSS: PortfolioComponent CRUD, sheet queries, column computation orchestration
spreadsheet-service	gRPC + REST	Go	HyperQL query execution, DimSlice/DimFold/DimExpand evaluation, HypercubeView management
space-service	gRPC + REST	Go	KSPC Space lifecycle, member management, governance config, treasury linking
workspace-service	gRPC + WebSocket	Go	KWSP Workspace sessions, sheet arrangements, presence, session history
link-network-service	gRPC + REST	Go + Rust	KLNK: LinkEdge CRUD, consent flows, ShadowRow provisioning, graph traversal API
crdt-service	gRPC	Rust	CrdtLog management, VectorClock advancement, merge operation application, conflict detection
identity-service	gRPC + REST	Go	KPID: SovereignEntity, Identity, Profile, Account management; VisibilityMask; CrossIdentityMerge
namespace-service	gRPC + REST	Go	KNSPC: NamespacePath resolution, alias management, federation sync, namespace health
writeback-service	gRPC	Go	AI engine result ingestion, permission validation, audit trail, cell write with System PermissionTier
export-service	REST (streaming)	Go	HG-EXPORT: CSV, XLSX, JSON-LD, Parquet, Arrow export; webhook delivery; embed generation
federation-coordinator	gRPC	Go	Cross-Grid CrossGridLink consent flows; shadow sync routing; cross-grid HyperQL fan-out

35. API Conventions

35.1 REST API Patterns

Pattern	Endpoint	Description
Entity CRUD	GET/POST/PUT/DELETE /api/v1/{domain}/{entity_type}/{id}	Standard CRUD for HyperRows within a domain cube
Sheet Query	GET /api/v1/{domain}/sheets/{sheet_id}?filter=...&sort=...&limit=50	Paginated sheet query with optional DimSlice predicates and HypercubeView config
HyperQL	POST /api/v1/hyperql	Execute an arbitrary HyperQL query against any accessible cube(s)
Graph Traversal	GET /api/v1/graph/traverse?from={id}&edge_type=...&depth=5	Traverse the Hypergraph from a starting node
Time Travel	GET /api/v1/{domain}/{entity_type}/{id}?as_of={timestamp}	AS_OF point-in-time entity state retrieval
Link Network	GET /api/v1/klnk/nodes/{id}/tree?depth=5&edge_type=...	KLNK link tree traversal from a given node
Namespace Resolution	GET /api/v1/ns/resolve?path=kogi://@alice/portfolio/...	Resolve a NamespacePath to its entity ID and Grid location
Shadow Row Sync	POST /api/v1/klnk/shadow/{shadow_id}/sync	Manually trigger MirrorAttribute sync from source entity
Export	POST /api/v1/export/sheets/{sheet_id}?format=parquet	Async export of a sheet or HypercubeView in the specified format
Natural Language Query	POST /api/v1/nl/query	NL-to-HyperQL translation and execution; returns structured results + generated HyperQL

35.2 WebSocket Real-Time Protocol (KPCL)

Real-time collaborative editing is powered by WebSocket connections managed by the KPCL service. All users viewing the same sheet are connected to the same CRDT broadcast channel. The protocol ensures all users see the same state within 150ms of any mutation.

Message Type	Direction	Description
subscribe	Client → Server	Subscribe to a sheet or Workspace for real-time updates. Server begins streaming CrdtOp events for that sheet.
crdt_op	Client → Server	Send a CRDT operation (SetField LWW, AddToSet OR-Set, etc.) originating from this client's user action.
crdt_op_broadcast	Server → Client	Broadcast a CRDT operation from any writer (including other users and AI engines) to all subscribed clients.

Message Type	Direction	Description
presence_update	Server → Client	Broadcast user presence: cursor position, active cell, focus status. Enables live cursor display.
conflict_notice	Server → Client	Notify client of a CRDT conflict that requires human attention (status lattice violation, cross-partition write).
oba_annotation	Server → Client	Push an Oba AI annotation for a specific row (proactive hint, anomaly flag, completion suggestion).
sync_complete	Server → Client	Confirm that all pending CrdtOps from this client have been applied and broadcast to all peers.

Part VIII — Open Items, Technical Decisions & Roadmap

36. Open Technical Decisions

Decision	Options Under Consideration	Current Recommendation	Target Milestone
N-dim index at scale (N≥4)	PostgreSQL composite index | Z-order curve | ClickHouse | Hybrid	Hybrid: PG for N≤4; ClickHouse for N>4 analytics cubes	v1.1
CRDT conflict surfacing UX	Silent (LWW always) | Notify (user review) | Block (require resolution)	Notify for semantic conflicts (Lattice); Silent for counters/sets	v1.0
Shadow cell write-back governance	Open (any editor) | Governed (proposal required) | Opt-in per attribute	Opt-in per attribute: write_back_attrs configurable per CrossGridLink	v1.0
Max N (dimensionality limit)	8 | 12 | 16 | Unlimited	Hard limit 16; recommended max N=6 for UI usability	v1.0 (finalize)
Formula engine sandboxing	WASM sandbox | Process isolation | Interpreted AST | JVM sandbox	Interpreted AST with resource limits (no arbitrary code execution)	v1.0
Federation identity model	Grid-level trust | Namespace-level trust | Identity-level trust | All three	All three tiers: Grid(full) > Namespace(partial) > Identity(minimal)	v1.5
AI engine default	OpenAI-compatible API | Local LLM (Ollama) | No default (platform-only)	Multiple: OpenAI-compatible + local LLM + no-AI mode configurable	v1.0
Status CRDT Lattice rollout	LWW (current) → Lattice (v2) | Lattice-only | Configurable per cube	LWW in v1 for simplicity; Lattice in v2 with migration tooling	v2.0
Global KLNK graph backend	PostgreSQL recursive CTE | Neo4j | Amazon Neptune | TigerGraph	PG recursive CTE for <1M nodes; Neo4j for platform-scale KLNK	v2.5
EventLog time-travel granularity	Day | Hour | Minute | Second | EventLog entry	EventLog entry: AS_OF resolves to last EventLog entry before timestamp	v1.0
PQL/HyperQL implementation backend	Custom parser + PG query generator | Presto/Trino | Apache Calcite	Custom parser + PG query generator (tight schema integration)	v1.0
AI Autonomous mode permissions	Per-action whitelist | Budget-capped autonomous | Time-limited autonomy	Per-action whitelist + budget cap (belt and suspenders safety)	v3.5

37. Priority Open Items

Item	Priority	System	Description	Blocks
Persistence Layer Pluggability	P0	All	PortfolioStore/SolutionStore trait abstraction for pluggable backends (SQLite, PostgreSQL, CouchDB). Currently PostgreSQL only.	Multi-node deployment; offline mode; edge devices
Status CRDT Lattice	P0	All	Implement custom lifecycle lattice for status merges. Currently status ops are LWW only; invalid concurrent state transitions possible.	Multi-node federated deployments
Space CRDT Sync	P0	Kogi/Ume	Extend CRDT federation to Space-scoped components. Space members on different nodes need consistent state.	Organization Space multi-node; Cooperative governance
Cross-Partition SplitPolicy	P0	Kogi	SplitPolicy defined but not enforced at Substrate layer for cross-partition CrdtOperations. Cross-partition edits bypass partition boundaries.	Multi-identity deployments; identity sovereignty
ShadowRow Real-Time Delta Sync	P1	Kogi/Qala	Kafka consumer for shadow sync; column-delta push instead of full row refresh. Currently batch-based. Performance critical at scale.	Cross-portfolio visibility at scale
EventLog Data Lake Flush	P1	All	10K event cap per entity with FIFO eviction risks historical loss. Plugin hook must flush to ClickHouse/S3 before eviction with full restoration path.	Long-lived component audit trail; AS_OF time-travel beyond hot window
WritebackService Permission Model	P1	All	Harden AI writer permission checks. Currently trust-based; needs cryptographic engine identity verification and per-key write authorization.	Production AI engine integration; security posture
Formula Engine v1	P1	Kogi/Qala	Full ColumnComputer expression language evaluator with all listed functions including RELATED(), GRAPH_NEIGHBORS(), and engine signal functions.	Custom column power users; Tier-1 computed attribute completeness
KLNK Consent Flow	P1	Kogi	Consent workflow for Collaborates/InvestedIn/Employs edge types (invitation, negotiation, column visibility configuration). Spec'd but unimplemented.	Cross-portfolio collaboration; economic graph formation

Item	Priority	System	Description	Blocks
HyperQL Full Implementation	P2	All	HyperQL parser and evaluator implemented against PostgreSQL; full JOIN support, SHADOW INCLUDE, AS_OF time-travel, TRAVERSE GRAPH, ROLLUP.	Advanced queries; analytics; cross-system joins
Real-Time Collaboration UI	P2	All	WebSocket-based live cursor indicators, cell lock indicators, presence badges, conflict resolution UX in sheet header.	Shared Workspace experience; KPCL user experience
Space Link Subgraph Materializ.	P2	Kogi	Hourly materialized view of SpaceLinkSubgraph per Space. Currently computed on demand; slow for large Spaces.	Space link visualization performance; large Community Spaces
Identity Merge Conflict Wizard	P2	Kogi	CrossIdentityMerge operation needs a UI-level conflict resolution wizard for cases where both source and target identities have rows with the same component_id.	Multi-identity power users; corporate/personal separation
KLNK Centrality Computation	P3	Kogi	Betweenness and eigenvector centrality computationally expensive at platform scale. Needs distributed approximation (sampled random walk) for graphs >1M nodes.	Platform-scale KLNK analytics; influence scoring
Template Marketplace	P3	All	KOGI-APPSTORE integration for distributing, rating, and purchasing portfolio/org/solution templates; template versioning and update notifications.	Template ecosystem; onboarding quality
On-Chain Identity Anchoring	P3	Kogi	DID (W3C Decentralized Identifier) anchoring for identity handles. Cross-platform reputation portability via verifiable credentials.	Web3 identity federation; cross-platform trust

38. Version Roadmap

Version	Milestone	Key Deliverables
v0.5 (Alpha)	Core Substrate	HyperCell model · N=2 dense encoding · AttributeKeyRegistry · EventLog · PostgreSQL persistence · Basic REST API · LWW CRDT · Single-node only
v1.0 (Beta)	2D+ Platform Launch	N=1–6 dimensions · Sparse + Hybrid encoding · HyperQL v1 · Formula language · View Engine (10 render modes) · Federation v1 · Spaces v1 · HG-NS v1 · Go SDK + TypeScript SDK · Kogi v3.0 core · Ume v1.0 core · Qala v1.0 core

Version	Milestone	Key Deliverables
v1.1	Full N-dim + Graph	All 16 dimension types · HG-GRAPH full implementation · CrossGridLink + ShadowCell protocol · LinkForest traversal · N-dim index optimization · ClickHouse analytics integration · Qala Domain Pack v1 (Software + Pharma)
v1.5	Intelligence + Identity	HG-AI pluggable engine · OpenAI-compatible adapter · NL-to-HyperQL bridge · AnomalyEngine · HG-ID multi-tenant identity · TenantPartition · VisibilityMask · CrossTenantMerge · Oba v1.0 Reactive + Proactive
v2.0	Full Platform	All 35 Kogi sheets · All Ume 42-module Hypercubes · Qala CCR/Release full governance · N-DimExplorer UI · Governance plugins (cooperative, democracy, multisig) · All export formats · Plugin marketplace · Status CRDT Lattice
v2.5	Scale + Federation	Neo4j graph backend for KLNK at platform scale · Cross-Grid AI (federated ML models) · Z-order curve multi-dim index · 255-node federation · GDPR/CCPA automated compliance workflows · KLNK centrality computation · Qala Domain Pack expansion (Financial, Medical Device, Research)
v3.0 (Kogi)	Autonomous Intelligence	Oba autonomous mode (approval-first) · Predictive scenario simulation · AI-generated portfolio narratives · Kogi v4.0 full feature set · KLNK v2 full graph API · Space federation (opt-in) · Federated MatchEngine
v3.5 (Kogi)	Web3 + Decentralization	On-chain identity anchoring (W3C DID) · Decentralized portfolio storage (IPFS + CouchDB) · Smart contract cooperative distribution · On-chain equity/cap table · Cross-federation KLNK traversal · DAO governance integration
v3.0 (Qala)	Factory Intelligence	Autonomous AI Agent (approval-first CCR automation) · Predictive DimFold quality models · Cross-Grid MatchEngine for talent and solution matching · Semantic layer (AI-generated cube descriptions + ontology mapping)
v4.0 (Platform)	Autonomous Platform	Cross-platform reputation portability via verifiable credentials · Platform-level autonomous governance · AI-generated platform roadmap proposals · Semantic interoperability between Kogi, Ume, and Qala ontologies · On-chain platform governance anchoring

Part IX — Appendices

Appendix A: Complete Glossary

Term	Definition
Apapo	The complete integrated platform ecosystem: Hypergrid substrate + Kogi IW-OS + Ume B-OS + Qala SF-OS. The name for the convergent whole.
Hypergrid	The N-Dimensional Distributed Spreadsheet System — the universal substrate. Provides all data infrastructure to domain systems.
Grid	Root container for all Hypercubes, dimensions, tenants, namespaces, spaces, and graph structures within one deployment.
Hypercube	An N-dimensional grid H=(D_1,D_2,…,D_N). The generalization of a spreadsheet sheet. Named {domain}.{entity_type} by convention.
HyperRow	All cells sharing the same D_1 key — the canonical entity in a Hypercube. The generalization of a spreadsheet row.
HyperCell	A data point at an N-dimensional coordinate, carrying an N-attribute map (AttributeMap). The generalization of a spreadsheet cell.
Universal Cell Store (UCS)	The physical storage layer holding all HyperCells across all Hypercubes. Keyed by (grid_id, cube_id, dim_keys_tuple).
DimensionAxis (D_i)	One axis of a Hypercube: EntityAxis (D_1), PropertyAxis (D_2), or any custom type (D_3–D_N).
AttributeMap	HashMap<AttributeKey, TypedAttrValue> — the N-attribute payload of a HyperCell.
AttributeKey	A registered string key in the AttributeKeyRegistry. The 'column name' generalized to apply across all dimensions.
TypedAttrValue	The typed value in a HyperCell attribute: Text | Number | Currency | Bool | DateTime | Enum | Json | Relation | Geo | Tag | User | Ai | Null.
DimSlice	A predicate applied to one or more dimension axes, producing a sub-cube or projection. Generalization of a row filter.
DimFold	Collapsing an axis by aggregating all its key values into a summary. Generalization of GROUP BY.
DimExpand	Expanding a dimension's key set as separate result columns. Generalization of PIVOT.
HypercubeView	A saved, shareable configuration of DimSlices, DimFolds, axis remappings, filters, sorts, and render mode.
HyperQL	The N-dimensional query language: SELECT, DimSlice WHERE, FOLD, EXPAND, TRAVERSE GRAPH, AS_OF, SHADOW INCLUDE.
Hypergraph	The graph layer: typed, directed edges between HyperCells, HyperRows, Cubes, Spaces, and Grids.

Term	Definition
CrossGridLink	A HypergraphEdge crossing Grid boundaries. Requires consent. Creates ShadowCells in the host Grid.
ShadowCell	A read-only reflection in Grid A of a linked entity from Grid B, created by a CrossGridLink.
MirrorAttribute	A specific attribute from a ShadowCell synced into the host Grid as a read-only computed attribute.
LinkForest	The complete set of all link trees rooted at a given identity — their full cross-grid network.
LinkTree	A rooted directed subgraph of the full Hypergraph from a single root entity to a configured max depth.
Space	A named, governed, bounded operational context within a Grid. Groups Hypercubes, members, and governance config.
Workspace	Active working session within a Space. Personalized Hypercube/view arrangement + session state.
NamespacePath	Hierarchical URI addressing any entity: {domain}://{space_type}/{slug}/...
SovereignEntity	The real-world entity (person, org, or agent) that owns a root set of Hypercubes and Spaces.
TenantPartition	A logical partition of a SovereignEntity's data, scoped to one Identity. Implemented via identity_tags on rows.
VisibilityMask	N-dimensional DimSlice predicates defining what is visible to each observer type (Public, Follower, Connection, Member, Owner).
SplitPolicy	Governance policy governing cross-partition data access, enforced at the Substrate layer.
VectorClock	HashMap<NodeId, u64>. Logical clock for causal ordering of all mutations across federation nodes.
CrdtLog	In-flight CRDT operation buffer: operations pending application and federation peer sync. Hot in Redis; durable in PostgreSQL.
EventLog	Append-only, immutable log of every mutation. Every cell attribute change is an EventLog entry.
AS_OF	Time-travel query operator: AS_OF timestamp returns cube state at that historical moment by replaying the EventLog.
Federation	CRDT synchronization between two or more Grid deployments. Delta sync via Kafka; VectorClock causal ordering.
HypercubePlugin	The extension interface for all Hypergrid customizations: axis types, attr types, AI engines, render modes, data connectors.
AIEngineAdapter	Plugin interface connecting any AI/ML service to provide Tier-2 computed attributes.
ComputedAttribute	A derived attribute: Tier-1 (synchronous formula) or Tier-2 (asynchronous AI/ML signal written via WritebackService).
WritebackService	The gRPC service accepting AI-computed values and writing them into HyperCells with full audit trail and permission enforcement.

Term	Definition
Domain Pack	A bundled HypercubePlugin collection for domain-specific governance, compliance, and AI in a regulated industry.
Kogi	Independent Worker OS. Root domain: Portfolio. Root Hyperspreadsheet: Portfolio System (KPMS).
KIMDSS	Kogi Interconnected Master Distributed Spreadsheet System — the complete Kogi platform.
KLNK	Kogi Link Network — the inter-portfolio graph: forests, trees, ShadowRows, MirrorColumns.
Oba	The Kogi platform AI assistant — the portfolio's AI chief-of-staff. Reactive, Proactive, and Autonomous modes.
kogi-engine	The Scala 3 intelligence engine consuming all portfolio events and writing back 20+ computed signals via WritebackService.
Portfolio	Kogi's root domain concept. The master spreadsheet of the independent worker's entire operational world. Not a feature — it is the OS.
Ume	Business OS. Root domain: Organization. Root Hyperspreadsheet: Organization System. 42 domain module Hypercubes.
Organization	Ume's root domain concept. The master spreadsheet of the organization's entire operational world.
Qala	Solution Factory OS. Root domain: Solution. Root Hyperspreadsheet: Solution System.
Solution	Qala's root domain concept and universal category. Kogi, Ume, and Qala itself are all Solutions in Qala's ontology.
SDE	Solution Development Environment — the primary governed workspace in Qala for building a solution.
CCR	Change Control Request — a formal governance record for a proposed change to a governed solution.
Root Factory	The Qala Root Factory that governs the Apapo platform itself. Kogi and Ume are Solutions in the Root Factory.
Domain Layer Pattern	The four-layer architecture used by all Apapo domain systems: Presentation → Domain Logic → Domain-Hypergrid Bridge → Hypergrid Substrate.

Appendix B: CRDT Selection Decision Tree

```
Is the attribute a multi-valued set (tags, members, dependencies, children, links)?
  YES → OR-Set
  NO ↓

Is the attribute a monotonically increasing counter (restart_count, view_count)?
  YES → GrowOnlyCounter
  NO ↓

Is the attribute a monotonically increasing maximum (version, sequence_number)?
```

```
  YES → MaxRegister
  NO ↓

Is the attribute a lifecycle state governed by a partial order (status,
lifecycle_state)?
  YES → Lattice (v2; LWW in v1 as interim)
  NO ↓

Is the attribute a numerical accumulator via transactions (budget_spent,
hours_logged)?
  YES → GrowOnlyCounter (if append-only) or LWW (if correctable)
  NO ↓

Is the attribute AI-computed (health_score, risk_score, regulatory_risk)?
  YES → LastWriteWins with PermissionTier::System write restriction
  NO ↓

Is the attribute human-authored content or a configuration value?
  YES → LastWriteWins
  NO → Consult domain architects; document the choice in AttributeKeyDef.notes
```

Appendix C: Key Design Decisions Log

Decision	Chosen Approach	Rationale	Decided
Primary entity storage layout	D_1=EntityId, D_2=FieldName, cell.attributes['value']	Maximizes HyperCell reuse; simplest codec pattern; uniform across all domain systems	v1.0
Portfolio as Kogi root domain	Portfolio IS the system — every entity is a row in the Portfolio	Philosophical and architectural clarity; simplifies cross-module data model; avoids duplication	v1.0
Organization as Ume root domain	Organization IS the system — every module entity is a row in the Org System	Same philosophical clarity as Kogi; 42 modules are views, not separate systems	v1.0
Solution as Qala root domain	Solution IS the universal category; factories, Kogi, and Ume are Solutions	Self-describing, self-governing architecture; maximum generalization of the root concept	v1.0
Status/lifecycle CRDT	LWW in v1; Lattice in v2	LWW is safe and simple to ship; Lattice adds governance enforcement at substrate level in v2	v1.0/v2.0
AI writeback mechanism	HypercubePlugin::compute_attribute + System PermissionTier	Consistent with plugin architecture; prevents user overwrite of AI scores; auditable	v1.0
Cross-system integration	CrossGridLink + ShadowCell + MirrorAttribute	Consent-gated; attribute-level granularity; no full data copy; respects data sovereignty	v1.0

Decision	Chosen Approach	Rationale	Decided
Federation sync protocol	CrdtLog delta sync over Apache Kafka	Reliable delivery; causal ordering; horizontally scalable; 255-node VectorClock limit	v1.0
Schema evolution policy	Additive changes are CRDT-commutative; destructive changes require gov gate	Zero-downtime schema updates; schema immutability as a governance invariant	v1.0
HyperQL vs standard SQL	HyperQL (custom N-dim) + NL bridge for user-facing queries	Native N-dim query support; AS_OF and TRAVERSE GRAPH are not expressible in standard SQL	v1.0
EventLog architecture	Dual-write: domain EventLog + HG EventLog	Unified audit trail + domain-typed querying; single source of truth for time-travel	v1.0
Namespace scheme	{domain}://{space_type}/{slug}/{entity_type}/{id}/	Domain prefixes prevent collision; URI-shaped for HTTP integration; human-readable addresses	v1.0
Max dimensionality	N=16 hard limit, N=6 practical recommendation	Beyond N=6, UI usability degrades significantly; N=16 covers all foreseeable edge cases	v1.0
Cell store encoding	Hybrid: dense for $D_1 \times D_2$, sparse for D_3+	Optimal performance for the common N=2 case while supporting higher-dim cubes efficiently	v1.0
AI computation tier separation	Tier-1 (sync formula) strictly separate from Tier-2 (async AI signal)	Separates fast, deterministic computations from slow, non-deterministic AI signals	v1.0

Appendix D: Comparative Analysis — Hypergrid vs. Conventional Systems

Category	System	Where It Overlaps with Hypergrid	Where Hypergrid Diverges	Honest Verdict
Relational DB	PostgreSQL, MySQL	Entity storage, ACID transactions, SQL queries, foreign key relationships	No CRDT semantics, no N-dimensional model, no native event sourcing, no graph layer, no AI computation. Schema changes require migrations.	PostgreSQL is Hypergrid's storage backend. For pure OLTP, use PostgreSQL directly. For domain systems that need CRDT + EventLog + Graph + AI, use Hypergrid on top.
Graph DB	Neo4j, Amazon Neptune	Property graph model, typed edges,	No rich N-dimensional property model, no	Neo4j is planned as Hypergrid's graph backend

Category	System	Where It Overlaps with Hypergrid	Where Hypergrid Diverges	Honest Verdict
		BFS/DFS traversal, shortest path	CRDT consistency, no time-travel, no AI attribute computation. Graph-only, not entity+graph.	at scale. For graphs <1M nodes, Hypergrid's built-in Hypergraph suffices. For larger graphs, Neo4j is used as a backend.
CRDT Store	Automerge, Yjs, Electric SQL	Conflict-free distributed editing, eventual consistency, collaborative sync	Designed for character-level collaborative text editing, not business data. No query language, no graph layer, no AI computation.	CRDT libraries solve character-level collaborative editing. Hypergrid uses CRDT semantics at the attribute key level — appropriate for business data. Not competing.
OLAP DB	ClickHouse, DuckDB, BigQuery	Aggregation, multi-dimensional slicing, column-oriented storage, DimFold/DimExpand operations	Read-optimized batch analytics only. No OLTP (individual reads/writes), no CRDT, no event sourcing, no graph layer.	ClickHouse is Hypergrid's analytics backend for high-volume DimFold workloads. Hypergrid is HTAP; ClickHouse is OLAP-only. They are complementary.
Event Store	EventStoreDB, Kafka	Append-only immutable event log, event sourcing, causal ordering	Pure event stores don't materialize current state — you always replay. Kafka is a transport, not a database. Neither has query language, graph layer, or AI.	Hypergrid takes the event sourcing insight (immutable EventLog) and integrates it into a full entity store where current state is always available without replay.
Spreadsheet	Excel, Google Sheets	Row/column model, formulas, filters, groups, pivot tables — the N=2 special case	2D only, not distributed, not versioned, no CRDT, no graph layer, no AI-computed attributes, no governance.	The conventional spreadsheet is the N=2 degenerate case of Hypergrid. Hypergrid generalizes the spreadsheet to N dimensions and makes it distributed, versioned, and AI-augmented.

Appendix E: Domain System Implementation Checklist

Use this checklist when building a new domain system on Hypergrid, following the same pattern used by Kogi, Ume, and Qala:

1. Define your root domain concept — the entity that is the center of gravity for everything in your system
2. Define all domain entities as pure Rust structs with no storage concerns
3. Register one Hypercube per major entity type using the naming convention {domain}.{entity_type}

4. Define all attribute keys with correct TypedAttrValue variants and CRDT semantics using the CRDT Decision Tree (Appendix B)
5. Configure PermissionTier for every attribute key based on who should be able to write it
6. Implement the DomainStore<Entity, EntityId> codec contract for each Hypercube
7. Register all Hypergraph EdgeType definitions that model relationships between your domain entities
8. Register AI engine plugins (AIEngineAdapter) for any Tier-2 computed attributes
9. Define your Space taxonomy using the HG-SPACE SpaceType enum and GovernanceConfig
10. Assign NamespacePath URI patterns for all entity types using domain-specific prefix (e.g., {domain}://...)
11. Define the CrossGridLink policy: which attribute keys may be mirrored from/to other domain systems?
12. Implement the WritebackService permission model for all AI-computed attributes
13. Write HyperQL queries for all common read patterns; test AS_OF time-travel for key entities
14. Define the EventLog dual-write configuration: domain event types + Hypergrid EventLog entry types
15. Test CRDT merge behavior for all concurrent write scenarios, especially status transitions
16. Optionally package domain-specific extensions as a Domain Pack (HypercubePlugin) for the APPSTORE

End of Document

Apapo Platform · Software Design Document · v1.0 · March 2026

Hypergrid Substrate · Kogi IW-OS · Ume B-OS · Qala SF-OS

Confidential — Internal Use Only